

TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT HÙNG YÊN

KHOA CÔNG NGHỆ THÔNG TIN

----- o0o -----

BÀI TẬP LỚN

CÔNG NGHỆ WEB VÀ ỨNG DỤNG

WEBSITE LẬP LỊCH VÀ QUẢN LÝ CÔNG VIỆC

GIẢNG VIÊN HƯỚNG DẪN: Ths. VŨ XUÂN THẮNG

SINH VIÊN: NGUYỄN XUÂN MONG

MÃ SINH VIÊN: 12423024

LỚP: 12423TN

HÙNG YÊN - 2026

NHẬN XÉT

Nhận xét của giảng viên:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Người hướng dẫn

LỜI CAM ĐOAN

Em cam kết rằng bài tập lớn “Website Lập Lịch và Quản Lý Công Việc” là kết quả do chính em thực hiện. Nội dung sử dụng trong bài tập không sao chép từ bất kỳ tài liệu nào. Các nội dung được trích dẫn đều được ghi đầy đủ nguồn và tôn trọng bản quyền. Những kết quả trình bày trong bài tập hoàn toàn là kết quả nghiên cứu cá nhân em dưới sự hướng dẫn của giảng viên. Nếu có bất kỳ sai sót nào, em xin hoàn toàn chịu trách nhiệm trước khoa và nhà trường.

Hung Yên, ngày 7 tháng 6 năm 2026

Sinh viên thực hiện

Nguyễn Xuân Mong

LỜI CẢM ƠN

Để có thể hoàn thành bài tập lớn này, trước hết em xin bày tỏ lòng biết ơn sâu sắc đến Khoa Công nghệ Thông tin - Trường Đại học Sư phạm Kỹ thuật Hưng Yên đã tạo điều kiện thuận lợi để em thực hiện bài tập lớn.

Đặc biệt, em xin gửi lời cảm ơn chân thành đến thầy Vũ Xuân Thắng đã tận tình hướng dẫn và chỉ bảo em trong suốt quá trình thực hiện. Những ý kiến chuyên môn quý báu cùng sự hỗ trợ liên tục của thầy đã góp phần quan trọng vào việc hoàn thành thành công đề tài này.

Em cũng xin chân thành cảm ơn toàn thể các thầy cô trong nhà trường đã tận tâm giảng dạy, trang bị cho em những kiến thức cần thiết và quý báu, giúp em có nền tảng vững chắc để thực hiện bài tập này. Sự tận tụy của các thầy cô trong công tác giảng dạy là nguồn động viên và cảm hứng lớn trong suốt quá trình học tập của em.

Mặc dù đã cố gắng hết sức, song do năng lực còn hạn chế, bài tập không tránh khỏi những thiếu sót trong quá trình thực hiện. Em rất mong nhận được những ý kiến đóng góp và nhận xét từ các thầy cô để có thể hoàn thiện hơn và rút ra nhiều kinh nghiệm quý báu.

Xin trân trọng cảm ơn các thầy cô đã dành thời gian, sự quan tâm và hướng dẫn cho em!

MỤC LỤC

NHẬN XÉT	1
LỜI CAM ĐOAN	2
LỜI CẢM ƠN.....	3
MỤC LỤC	4
DANH MỤC HÌNH ẢNH.....	7
DANH MỤC BẢNG	8
CHƯƠNG 1: GIỚI THIỆU	9
1.1. Bối cảnh và vấn đề.....	9
1.1.1. Bối cảnh thực tiễn	9
1.1.2. Các vấn đề người dùng gặp phải.....	9
1.2 Lý do chọn đề tài.....	10
1.3. Phạm vi và giới hạn.....	11
1.4. Bố cục báo cáo	12
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT.....	13
2.1. Tổng quan chương 2.....	13
2.2. ReactJS	13
2.2.1. Giới thiệu	13
2.2.2. Các khái niệm cốt lõi	13
2.2.3. Virtual DOM.....	17
2.3. Node.js.....	18
2.3.1. Giới thiệu	18
2.3.2. Các khái niệm cốt lõi	18
2.4. Express.js.....	19

Website Lập Lịch và Quản Lý Công Việc	
2.4.1. Giới thiệu	19
2.4.2. Các khái niệm cốt lõi	20
2.5. MySQL	24
2.5.1. Giới thiệu	24
2.5.2. Các khái niệm cốt lõi	24
2.6.1. Giới thiệu	25
2.6.2. Nguyên tắc thiết kế RESTful API	26
2.7. JWT (JSON Web Token)	26
2.7.1. Giới thiệu	26
2.7.2. Cấu trúc JWT	26
2.8. Zustand (State Management)	27
2.8.1. Giới thiệu	27
2.8.2. Cú pháp cơ bản	27
2.9. TanStack Query (React Query)	28
2.9.1. Giới thiệu	28
2.9.2. Cú pháp cơ bản	28
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG	30
3.1 Tổng quan chương 3.....	30
3.2. Phân tích bài toán	31
3.2.1.Mô tả bài toán chi tiết và logic	31
3.2.2. Yêu cầu chức năng chi tiết.....	33
3.2.3. Yêu cầu phi chức năng (Non-Functional Requirements)	37
3.3. Phân tích Use Case	39
3.3.1. Biểu đồ Use Case tổng quan	39
3.3.2. Đặc tả chi tiết các Use Case quan trọng	40
3.4. Biểu đồ tuần tự (Sequence Diagram).....	51

Website Lập Lịch và Quản Lý Công Việc	
3.4.1. Quy trình Đăng nhập hệ thống	52
3.4.2. Quy trình Tạo công việc mới	53
3.4.3. Quy trình Đồng bộ email tự động nền.....	55
3.4.4. Quy trình Chat và Lập kế hoạch với AI.....	57
3.4.5. Quy trình Nhắc nhở qua Discord	60
3.4.6. Quy trình Quản lý Workspace và Giao việc nhóm.....	61
3.5. Thiết kế cơ sở dữ liệu	63
3.5.1. Danh sách các bảng dữ liệu	63
3.5.2. Chi tiết cấu trúc từng bảng dữ liệu	63
3.6. Thiết kế kiến trúc hệ thống.....	68
3.6.1. Kiến trúc 3 tầng (3-tier Architecture)	68
CHƯƠNG 4: KẾT QUẢ TRIỂN KHAI WEBSITE	70
4.1 Môi trường triển khai.....	70
4.2 Kết quả giao diện các trang chính.....	70
4.3 Kết quả chức năng tự động hóa nền	76
4.4 Kết quả tích hợp AI.....	77
KẾT LUẬN	79
TÀI LIỆU THAM KHẢO	80

DANH MỤC HÌNH ẢNH

Hình 3. 1: Sơ đồ minh họa cho Mô tả bài toán chi tiết và logic.....	31
Hình 3. 2: Sơ đồ minh họa cho Biểu đồ Use Case tổng quan	39
Hình 3. 3: Sơ đồ minh họa cho Quy trình Đăng nhập hệ thống.....	52
Hình 3. 4: Sơ đồ minh họa cho Quy trình Tạo công việc mới	54
Hình 3. 5: Sơ đồ minh họa cho Quy trình Đồng bộ email tự động nền	56
Hình 3. 6: Sơ đồ minh họa cho Quy trình Chat và Lập kế hoạch với AI.....	58
Hình 3. 7: Sơ đồ minh họa cho Quy trình Nhắc nhở qua Discord	60
Hình 3. 8: Sơ đồ minh họa cho Quy trình Quản lý Workspace và Giao việc nhóm	62
Hình 3. 9: Sơ đồ minh họa cho Biểu đồ quan hệ thực thể (ERD)	67
Hình 3. 10: Sơ đồ minh họa cho Kiến trúc 3 tầng (3-tier Architecture).....	68
Hình 4. 1: Giao diện trang Đăng nhập.....	71
Hình 4. 2: Giao diện trang Dashboard.....	71
Hình 4. 3: Giao diện trang Quản lý công việc	72
Hình 4. 4: Giao diện trang Lịch.....	73
Hình 4. 5: Giao diện trang Phân tích hiệu suất.....	73
Hình 4. 6: Giao diện trang Trợ lý AI	74
Hình 4. 7: Giao diện trang Không gian làm việc nhóm	75
Hình 4. 8: Giao diện trang Cài đặt.....	75
Hình 4. 9: Log kết quả đồng bộ email tự động.....	76
Hình 4. 10: Thông báo nhắc nhở deadline qua Discord	76
Hình 4. 11: AI đề xuất lịch trình dạng thẻ kế hoạch	77
Hình 4. 12: Kết quả AI trích xuất task từ nội dung email	77

DANH MỤC BẢNG

Bảng 3. 1: Bảng nội dung trong mục Đối tượng sử dụng hệ thống.....	32
Bảng 3. 2: Bảng nội dung trong mục Yêu cầu phi chức năng (Non-Functional Requirements).....	37
Bảng 3. 3: Bảng nội dung trong mục Use Case: Đăng nhập hệ thống	41
Bảng 3. 4: Bảng nội dung trong mục Use Case: Tạo công việc mới	42
Bảng 3. 5: Bảng nội dung trong mục Use Case: Chat tương tác với trợ lý AI.....	44
Bảng 3. 6: Bảng nội dung trong mục Use Case: Đồng bộ email tự động nền.....	46
Bảng 3. 7: Bảng nội dung trong mục Use Case: AI đề xuất lịch trình & Duyệt kế hoạch	48
Bảng 3. 8: Bảng nội dung trong mục Use Case: Nhắc nhở chủ động qua Discord..	50
Bảng 3. 9: Bảng nội dung trong mục Danh sách các bảng dữ liệu	63
Bảng 3. 10: Bảng nội dung trong mục Bảng users (Thông tin người dùng)	64
Bảng 3. 11: Bảng nội dung trong mục Bảng tasks (Thông tin công việc)	64
Bảng 3. 12: Bảng nội dung trong mục Bảng events (Sự kiện lịch hẹn)	65
Bảng 3. 13: Bảng nội dung trong mục Bảng notifications (Nhật ký thông báo hệ thống)	66

CHƯƠNG 1: GIỚI THIỆU

1.1. Bối cảnh và vấn đề

1.1.1. Bối cảnh thực tiễn

Trong thời đại số hóa và làm việc từ xa phát triển mạnh mẽ như hiện nay, việc quản lý thời gian và công việc cá nhân trở nên ngày càng phức tạp. Người dùng — đặc biệt là sinh viên, nhân viên văn phòng và người làm việc tự do — hàng ngày phải tiếp nhận một lượng thông tin khổng lồ đổ về từ nhiều nguồn khác nhau: email từ trường học hoặc công ty, tin nhắn công việc từ các nhóm Discord, Slack, Zalo, cùng với các thông báo từ nhiều ứng dụng khác nhau.

Để quản lý tất cả các đầu việc này, đa số mọi người buộc phải sử dụng đồng thời từ 5 đến 7 công cụ rời rạc: Google Calendar để xem lịch họp, Notion để ghi chép, Trello để theo dõi tiến độ, Microsoft To Do để lập danh sách việc cần làm... Việc liên tục chuyển đổi ngữ cảnh và sao chép dữ liệu giữa các nền tảng này không chỉ làm giảm năng suất cá nhân mà còn trực tiếp gây ra tình trạng bỏ sót công việc quan trọng, trễ hạn nộp bài hoặc báo cáo, từ đó làm tăng mức độ căng thẳng và giảm hiệu quả làm việc tổng thể.

1.1.2. Các vấn đề người dùng gặp phải

- **Vấn đề 1: Phân tán nguồn công việc**
 - Deadline bài tập đến qua email trường, yêu cầu chỉnh sửa báo cáo qua Discord, lịch họp nhóm qua Zalo — tất cả nằm rải rác trên các kênh khác nhau mà không có nơi tổng hợp. Hệ quả là người dùng thường xuyên bỏ sót công việc quan trọng chỉ vì chúng bị chôn vùi trong hàng trăm tin nhắn chưa đọc.
- **Vấn đề 2: Thiếu công cụ lên lịch thông minh**
 - Các ứng dụng hiện tại về bản chất chỉ là kho lưu trữ tĩnh — chứa danh sách việc cần làm nhưng không thể gợi ý thứ tự ưu tiên hay sắp xếp lịch trình hợp lý. Người dùng vẫn phải tự lên kế hoạch hoàn toàn thủ công, đặc biệt bất tiện khi có nhiều việc quan trọng cùng lúc hoặc xuất hiện việc đột xuất.
- **Vấn đề 3: Thiếu cơ chế nhắc nhở chủ động**

Website Lập Lịch và Quản Lý Công Việc

- Thông báo in-app hay email nhắc nhở dễ bị bỏ lỡ khi tab trình duyệt đã đóng hoặc người dùng đang tập trung làm việc — đúng lúc cần được nhắc nhở nhất là lúc ít chú ý đến thông báo nhất.
- **Vấn đề 4: Không đo lường được hiệu suất cá nhân**
 - Người dùng không có dữ liệu khách quan để tự đánh giá: tuần này hoàn thành bao nhiêu việc, loại task nào hay bị trễ, hay thời điểm nào làm việc hiệu quả nhất. Sự thiếu vắng phản hồi định lượng này khiến việc cải thiện thói quen làm việc trở nên mơ hồ và thiếu căn cứ.
- **Vấn đề 5: Thiếu không gian cộng tác nhóm**
 - Việc phân công và theo dõi tiến độ nhóm thường phụ thuộc vào Trello, Google Sheets hoặc nhắn tin thủ công — hoàn toàn tách biệt khỏi không gian quản lý công việc cá nhân. Người dùng lại rơi vào vòng lặp phải dùng nhiều công cụ rời rạc song song.

1.2 Lý do chọn đề tài

Xuất phát từ những vấn đề thực tiễn trên, đề tài **"FocusFlow — Hệ thống quản lý lịch trình và công việc cá nhân tích hợp AI"** được thực hiện nhằm xây dựng một giải pháp toàn diện, tập trung và thông minh hơn so với các sản phẩm hiện có trên thị trường.

Qua nghiên cứu các ứng dụng phổ biến như Todoist, Microsoft To Do hay Google Tasks, nhận thấy một điểm chung: chúng đều hoạt động thụ động — người dùng phải tự nhập, tự theo dõi và tự nhắc nhở bản thân. Chúng thiếu khả năng tự động hóa đầu vào, thiếu tích hợp AI thực sự hữu ích và không hỗ trợ cộng tác nhóm trong cùng một không gian quản lý. FocusFlow được thiết kế để lấp đầy chính xác những khoảng trống này với các điểm khác biệt nổi bật:

- **Tự động hóa đầu vào từ email:** Hệ thống tự động quét hộp thư qua giao thức IMAP và sử dụng AI để trích xuất thông tin công việc, tạo task trực tiếp vào hệ thống mà không cần người dùng thao tác thủ công.
- **Trợ lý AI lên kế hoạch bằng tiếng Việt:** Người dùng chỉ cần mô tả nhu cầu bằng ngôn ngữ tự nhiên, trợ lý AI sẽ phân tích ngữ cảnh danh sách công việc

Website Lập Lịch và Quản Lý Công Việc

hiện có và đề xuất kế hoạch tối ưu dưới dạng thẻ trực quan để xác nhận hàng loạt.

- **Nhắc nhở chủ động qua Discord:** Thay vì dựa vào thông báo in-app dễ bị bỏ qua, hệ thống tự động gửi cảnh báo deadline trước 30 phút vào kênh Discord cá nhân — nền tảng mà nhóm đối tượng mục tiêu sử dụng liên tục hàng ngày.
- **Cộng tác và giao nhận việc theo nhóm:** FocusFlow hỗ trợ tạo workspace nhóm, mời thành viên, phân quyền và giao việc cho từng người — tích hợp liền mạch với không gian quản lý công việc cá nhân trong cùng một ứng dụng.
- **Phân tích hiệu suất trực quan:** Hệ thống tự động thống kê và trực quan hóa dữ liệu công việc theo tuần, theo mức độ ưu tiên và theo nguồn gốc, cung cấp cho người dùng góc nhìn khách quan về năng suất của bản thân.

Ngoài giá trị thực tiễn, đề tài còn là cơ hội để nghiên cứu và áp dụng tổng hợp các công nghệ hiện đại bao gồm ReactJS, Node.js, MySQL, kỹ thuật Prompt Engineering với mô hình ngôn ngữ lớn (LLM), cùng các giao thức tích hợp thực tế như IMAP, Google OAuth 2.0 và Discord Webhook.

1.3. Phạm vi và giới hạn

- Phạm vi chức năng:

Hệ thống FocusFlow bao gồm các phân hệ chức năng chính sau:

- **Quản lý tài khoản:** Đăng ký, đăng nhập bằng tài khoản hệ thống hoặc Google OAuth 2.0, quản lý hồ sơ và bảo mật phiên bằng JWT.
- **Quản lý công việc cá nhân:** Thêm, sửa, xóa, hoàn thành công việc; gán mức độ ưu tiên, đặt deadline; lọc theo ngày hoặc theo nguồn gốc (Custom, Email, AI).
- **Dashboard và lịch trình:** Tổng hợp tiến trình công việc trong ngày dạng Timeline; lịch tháng tích hợp Dương lịch — Âm lịch và ngày nghỉ lễ Việt Nam.
- **Workspace và cộng tác nhóm:** Tạo không gian làm việc chung, mời thành viên qua email, phân quyền Owner/Admin/Member, giao việc cho từng thành viên và theo dõi tiến độ nhóm.
- **Trợ lý AI:** Chatbot lên kế hoạch bằng tiếng Việt tự nhiên, phân tích danh sách công việc hiện có và xuất kế hoạch dạng thẻ trực quan để xác nhận hàng loạt.

Website Lập Lịch và Quản Lý Công Việc

- **Đồng bộ email tự động:** Background worker quét email qua IMAP mỗi 5 phút, lọc thư công việc và gọi AI trích xuất thông tin tạo task tự động.
- **Nhắc nhở qua Discord:** Tự động gửi thông báo Markdown cảnh báo deadline trước 30 phút vào Discord cá nhân của người dùng.
- **Phân tích hiệu suất:** Biểu đồ SVG thống kê số task hoàn thành theo tuần, phân bổ theo mức độ ưu tiên và nguồn gốc công việc.
- **Dọn dẹp tự động:** Background worker chạy mỗi giờ, tự động xóa dữ liệu cũ hơn 30 ngày để giải phóng tài nguyên hệ thống.

- Giới hạn của đề tài

Do giới hạn về thời gian và nguồn lực, đề tài hiện chưa hỗ trợ một số tính năng sau:

- Đồng bộ email qua Gmail API hay Microsoft Graph API; hệ thống hiện chỉ hỗ trợ IMAP truyền thống.
- Thông báo qua các kênh ngoài Discord như Telegram, Zalo hay SMS.
- Chế độ hoạt động ngoại tuyến (offline mode).
- Ứng dụng di động native; giao diện hiện được tối ưu cho Desktop và Tablet.

1.4. Bố cục báo cáo

Báo cáo được cấu trúc thành 4 chương như sau:

- **Chương 1 — Giới thiệu:** Trình bày bối cảnh thực tiễn, phân tích các vấn đề người dùng gặp phải, lý do chọn đề tài, phạm vi và giới hạn của hệ thống.
- **Chương 2 — Cơ sở lý thuyết:** Trình bày các khái niệm nền tảng và công nghệ cấu thành hệ thống, bao gồm ReactJS, Node.js, MySQL, LLM và các giao thức tích hợp.
- **Chương 3 — Phân tích và thiết kế hệ thống:** Mô tả kiến trúc hệ thống, sơ đồ Use Case, các luồng tương tác chính và thiết kế cơ sở dữ liệu quan hệ.
- **Chương 4 — Kết quả triển khai:** Minh họa giao diện ứng dụng, kết quả hoạt động của các tiến trình tự động và định hướng phát triển trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan chương 2

Để xây dựng hệ thống FocusFlow hoạt động ổn định và đáp ứng các yêu cầu chức năng lẫn phi chức năng, một tập hợp công nghệ, giao thức mạng và mô hình kiến trúc hiện đại đã được nghiên cứu và lựa chọn kỹ lưỡng. Chương này trình bày cơ sở lý thuyết và cú pháp cơ bản của các công nghệ cốt lõi cấu thành hệ thống, bao gồm: ReactJS kết hợp Zustand và TanStack Query; Node.js và Express.js với cơ chế xác thực JWT và mã hóa bcryptjs; MySQL; các giao thức tích hợp ngoại vi (IMAP, Discord Webhook, Google OAuth 2.0); và mô hình Client-Server.

2.2. ReactJS

2.2.1. Giới thiệu

ReactJS là thư viện JavaScript mã nguồn mở do Facebook (nay là Meta) phát triển năm 2013, dùng để xây dựng giao diện người dùng (UI) cho ứng dụng web theo mô hình Component-Based Architecture — giao diện được chia thành các thành phần nhỏ, độc lập và có thể tái sử dụng. Tính đến nay, React là thư viện frontend phổ biến nhất thế giới với hơn 220.000 stars trên GitHub, được sử dụng bởi Facebook, Instagram, Netflix, Airbnb và Uber.

2.2.2. Các khái niệm cốt lõi

a) JSX (JavaScript XML)

JSX là cú pháp mở rộng cho JavaScript, cho phép viết cấu trúc HTML trực tiếp bên trong code JavaScript. JSX không phải HTML thuần — trình biên dịch sẽ chuyển đổi nó thành các lệnh gọi hàm JavaScript trước khi trình duyệt thực thi.

Cú pháp cơ bản:

```
const element = <h1>Xin chào, FocusFlow!</h1>;
```

```
const name = "Người dùng";
```

```
const greeting = <h1>Xin chào, {name}!</h1>;
```

```
const      button      =      <button      className="btn-primary"
onClick={handleClick}>Bấm</button>;
```

Giải thích:

- `{name}`: Cặp ngoặc nhọn cho phép nhúng bất kỳ biểu thức JavaScript nào (biến, phép tính, lời gọi hàm).
- `className`: Trong JSX, thuộc tính class của HTML được đổi thành `className` vì class là từ khóa dành riêng trong JavaScript.
- `onClick={handleClick}`: Sự kiện trong JSX dùng cú pháp camelCase thay vì lowercase như HTML thuần.

b) Component (Thành phần giao diện)

Component là khối xây dựng cơ bản của React. Mỗi component là một hàm JavaScript nhận đầu vào (props) và trả về JSX mô tả giao diện.

Cú pháp cơ bản:

```
function TaskCard({ title, priority, dueDate }) {  
  return (  
    <div className="task-card">  
      <h3>{title}</h3>  
      <span>Ưu tiên: {priority}</span>  
      <span>Hạn: {dueDate}</span>  
    </div>  
  );  
}
```

// Sử dụng component:

```
<TaskCard title="Nộp báo cáo" priority="Cao" dueDate="30/05/2026" />
```

Giải thích:

- `function TaskCard({ title, priority, dueDate })`: Định nghĩa component tên TaskCard. Cú pháp `{ title, priority, dueDate }` là destructuring — tách các thuộc tính riêng lẻ từ đối tượng props để sử dụng trực tiếp.

Website Lập Lịch và Quản Lý Công Việc

- `return (...)`: Trả về JSX. Nếu JSX có nhiều dòng, bọc trong cặp ngoặc tròn `()` để tránh lỗi cú pháp.
- Khi sử dụng component, truyền dữ liệu qua props giống như thuộc tính HTML.

c) State (Trạng thái nội bộ)

State là dữ liệu nội bộ của component, có thể thay đổi theo thời gian. Khi state thay đổi, React tự động render lại component để cập nhật giao diện.

Cú pháp cơ bản:

```
import { useState } from 'react';

function Counter() {

  const [count, setCount] = useState(0);

  return (

    <div>

      <p>Số đếm: {count}</p>

      <button onClick={() => setCount(count + 1)}>Tăng</button>

    </div>

  );

}
```

Giải thích:

- `useState(0)`: Hook khai báo state với giá trị khởi tạo ban đầu là 0.
- `const [count, setCount]`: Destructuring mảng — `count` là giá trị state hiện tại, `setCount` là hàm dùng để cập nhật.
- `setCount(count + 1)`: Gọi hàm setter với giá trị mới. React tự động so sánh giá trị cũ và mới, nếu khác nhau thì render lại component.
- Nguyên tắc quan trọng: KHÔNG BAO GIỜ gán trực tiếp `count = 5`, phải luôn dùng `setCount(5)` để React nhận biết sự thay đổi.

d) useEffect Hook (Hiệu ứng phụ)

Website Lập Lịch và Quản Lý Công Việc

useEffect cho phép thực hiện các "hiệu ứng phụ" (side effects) trong component như gọi API, đăng ký sự kiện, thao tác DOM hoặc thiết lập timer.

Cú pháp cơ bản:

```
import { useState, useEffect } from 'react';

function TaskList() {

  const [tasks, setTasks] = useState([]);

  useEffect(() => {

    fetch('/api/tasks')

      .then(res => res.json())

      .then(data => setTasks(data));

  }, []);

  return (

    <ul>

      {tasks.map(task => <li key={task.id}>{task.title}</li>)}

    </ul>

  );

}
```

Giải thích:

- `useEffect(() => { ... }, [])`: Hook thực hiện side effect sau khi component được render lên màn hình.
- Dependency array quyết định khi nào effect chạy lại: `[]` (mảng rỗng) chỉ chạy 1 lần sau lần render đầu tiên; `[count]` chạy lại mỗi khi biến `count` thay đổi; không truyền thì chạy sau mỗi lần render.
- `key={task.id}`: Thuộc tính đặc biệt giúp React nhận diện và tối ưu việc render danh sách.

e) Props (Thuộc tính truyền vào)

Website Lập Lịch và Quản Lý Công Việc

Props là cơ chế truyền dữ liệu một chiều từ component cha sang component con. Props là read-only — component con không được phép thay đổi props nhận được.

Cú pháp cơ bản:

```
function App() {  
  
  return <TaskCard title="Nộp báo cáo" priority={1} isDone={false} onComplete={()  
=> alert("Xong!")} />;  
  
}  
  
function TaskCard({ title, onComplete }) {  
  
  return (  
  
    <div>  
  
      <h3>{title}</h3>  
  
      <button onClick={onComplete}>Hoàn thành</button>  
  
    </div>  
  
  );  
  
}
```

Giải thích:

- Giá trị không phải string phải bọc trong {}: priority={1}, isDone={false}.
- Có thể truyền hàm callback xuống component con: onComplete={() => alert('Xong!)}.
- Luồng dữ liệu một chiều (one-way data flow): Cha → Con. Con muốn gửi dữ liệu ngược lại phải thông qua callback.

2.2.3. Virtual DOM

React sử dụng Virtual DOM (DOM ảo) để tối ưu hiệu suất. Thay vì thao tác trực tiếp trên DOM thật, React duy trì một bản sao nhẹ của DOM trong bộ nhớ. Khi state thay đổi, React tạo Virtual DOM mới, so sánh với bản cũ bằng thuật toán Diffing để tìm điểm khác biệt, rồi chỉ cập nhật những phần thay đổi trên DOM thật (Reconciliation). Nhờ đó, React đạt hiệu suất cao ngay cả khi giao diện phức tạp.

2.3. Node.js

2.3.1. Giới thiệu

Node.js là môi trường thực thi JavaScript phía máy chủ (server-side runtime), được xây dựng trên V8 JavaScript Engine của Google Chrome. Node.js cho phép dùng JavaScript để viết mã phía server, xử lý logic nghiệp vụ, kết nối cơ sở dữ liệu và xây dựng API. Node.js được tạo bởi Ryan Dahl năm 2009 với các đặc điểm nổi bật: Non-blocking I/O (xử lý nhiều request đồng thời mà không cần tạo thread mới), Event-Driven Architecture (dùng Event Loop xử lý bất đồng bộ hiệu quả), và hệ sinh thái NPM với hơn 2 triệu package.

2.3.2. Các khái niệm cốt lõi

a) Module System (Hệ thống module)

Node.js sử dụng hệ thống module để tổ chức code thành các file riêng biệt. Hệ thống CommonJS (mặc định) dùng `require()` để nhập và `module.exports` để xuất.

```
// File: mathUtils.js
```

```
function add(a, b) { return a + b; }
```

```
module.exports = { add };
```

```
// File: app.js
```

```
const { add } = require('./mathUtils');
```

```
console.log(add(2, 3)); // Output: 5
```

Giải thích:

- `module.exports = { add };`: Xuất hàm `add` ra ngoài module để các file khác có thể truy cập.
- `const { add } = require('./mathUtils');`: Nhập hàm `add` từ file `mathUtils.js`. Đường dẫn `./` biểu thị file cùng thư mục.

b) Async/Await (Xử lý bất đồng bộ)

Node.js xử lý I/O theo cơ chế bất đồng bộ. `async/await` là cú pháp hiện đại giúp viết code bất đồng bộ trông giống code đồng bộ, dễ đọc và dễ bảo trì hơn so với `callback` hoặc `.then().catch()`.

Website Lập Lịch và Quản Lý Công Việc

```
async function getTasks() {  
  
  try {  
  
    const [rows] = await pool.execute('SELECT * FROM tasks WHERE user_id = ?',  
[1]);  
  
    console.log('Có', rows.length, 'công việc');  
  
    return rows;  
  
  } catch (error) {  
  
    console.error('Lỗi:', error.message);  
  
  }  
  
}
```

Giải thích:

- `async function`: Khai báo hàm bất đồng bộ, luôn trả về một Promise.
- `await`: Chỉ dùng được trong hàm `async`. Tạm dừng thực thi hàm cho đến khi Promise trả về kết quả — thành công thì trả về giá trị, thất bại thì ném lỗi để `catch` bắt.
- `try...catch`: Bắt mọi lỗi xảy ra trong khối `try`.

c) Event Loop (Vòng lặp sự kiện)

Event Loop là cơ chế nền tảng giúp Node.js xử lý hàng nghìn kết nối đồng thời chỉ với một thread duy nhất. Khi một thao tác I/O được gọi, Node.js đẩy nó vào hàng đợi và tiếp tục xử lý request tiếp theo. Khi I/O hoàn thành, callback tương ứng được đưa vào Event Loop để thực thi.

2.4. Express.js

2.4.1. Giới thiệu

Express.js là web framework nhẹ và linh hoạt cho Node.js, dùng để xây dựng ứng dụng web và RESTful API. Express cung cấp bộ công cụ tối giản nhưng mạnh mẽ, giúp thiết lập server, định nghĩa route, xử lý request/response và tích hợp middleware. Đây là framework backend phổ biến nhất trong hệ sinh thái Node.js với hơn 65.000 stars trên GitHub.

2.4.2. Các khái niệm cốt lõi

a) Tạo ứng dụng Express cơ bản

```
const express = require('express');

const app = express();

app.use(express.json());

app.get('/api/hello', (req, res) => {

  res.json({ message: 'Xin chào từ FocusFlow API!' });

});

const PORT = 5000;

app.listen(PORT, () => console.log(`Server đang chạy tại: http://localhost:${PORT}`));
```

Giải thích:

- `const app = express()`: Tạo instance ứng dụng Express với các phương thức để định nghĩa route, dùng middleware và cấu hình server.
- `app.use(express.json())`: Đăng ký middleware toàn cục, tự động parse body JSON thành đối tượng JavaScript trên `req.body` cho mọi request.
- `res.json(...)`: Gửi phản hồi dạng JSON về client, tự động set header `Content-Type: application/json`.

b) Route và HTTP Methods

Route xác định ứng dụng phản hồi như thế nào khi nhận request đến một endpoint (đường dẫn + phương thức HTTP).

// GET - Lấy danh sách công việc

```
app.get('/api/tasks', async (req, res) => {

  const [tasks] = await pool.execute('SELECT * FROM tasks');

  res.json(tasks);

});
```

// POST - Tạo công việc mới

Website Lập Lịch và Quản Lý Công Việc

```
app.post('/api/tasks', async (req, res) => {  
  
  const { title, priority, due_date } = req.body;  
  
  await pool.execute('INSERT INTO tasks (title, priority, due_date) VALUES (?, ?, ?)',  
[title, priority, due_date]);  
  
  res.status(201).json({ message: 'Đã tạo công việc' });  
  
});  
  
// PUT - Cập nhật công việc  
  
app.put('/api/tasks/:id', async (req, res) => {  
  
  const { id } = req.params;  
  
  const { title, status } = req.body;  
  
  await pool.execute('UPDATE tasks SET title = ?, status = ? WHERE id = ?', [title,  
status, id]);  
  
  res.json({ message: 'Đã cập nhật' });  
  
});  
  
// DELETE - Xóa công việc  
  
app.delete('/api/tasks/:id', async (req, res) => {  
  
  const { id } = req.params;  
  
  await pool.execute('DELETE FROM tasks WHERE id = ?', [id]);  
  
  res.json({ message: 'Đã xóa' });  
  
});
```

Giải thích:

- HTTP Methods thể hiện hành động: GET đọc dữ liệu; POST tạo mới; PUT cập nhật (ghi đè); DELETE xóa tài nguyên.
- '/api/tasks/:id': Dấu :id là route parameter — biến động trong URL, truy cập qua req.params.id.
- req.body: Chứa dữ liệu từ body request (chỉ có ở POST, PUT, PATCH), cần middleware express.json() để parse.

Website Lập Lịch và Quản Lý Công Việc

- req.query: Chứa tham số truy vấn từ query string, ví dụ /api/tasks?status=done
→ req.query.status = 'done'.

c) Middleware (Phần mềm trung gian)

Middleware là hàm có quyền truy cập vào req, res và hàm next(). Middleware xử lý request trước khi đến route handler, dùng để thêm các tính năng xuyên suốt như xác thực, logging hay CORS.

```
Function authMiddleware(req, res, next) {  
  
  const token = req.headers.authorization?.split(' ')[1];  
  
  if (!token) return res.status(401).json({ message: 'Chưa đăng nhập' });  
  
  try {  
  
    const decoded = jwt.verify(token, process.env.JWT_SECRET);  
  
    req.user = decoded;  
  
    next();  
  
  } catch (error) {  
  
    return res.status(401).json({ message: 'Token không hợp lệ' });  
  
  }  
  
}
```

```
app.use('/api/tasks', authMiddleware, taskRoutes);
```

Giải thích:

- Middleware nhận 3 tham số: req (request), res (response) và next (hàm gọi middleware tiếp theo). Nếu không gọi next(), request sẽ bị “treo”.
- req.headers.authorization?.split(' ')[1]: Trích xuất token từ header dạng “Bearer <token>”. Toán tử ?. (optional chaining) tránh lỗi nếu header không tồn tại.
- req.user = decoded: Gắn thông tin user (id, username) vào request để route handler phía sau sử dụng.
- app.use('/api/tasks', authMiddleware, taskRoutes): Áp dụng middleware cho tất cả route bắt đầu bằng /api/tasks.

d) Router (Bộ định tuyến)

Express Router cho phép tách route thành các file riêng biệt, giúp code sạch sẽ và dễ bảo trì.

```
// File: routes/taskRoutes.js

const express = require('express');

const router = express.Router();

const taskController = require('../controllers/taskController');

router.get('/', taskController.getTasks);

router.post('/', taskController.createTask);

router.put('/:id', taskController.updateTask);

router.delete('/:id', taskController.deleteTask);

module.exports = router;

// File: server.js

const taskRoutes = require('./routes/taskRoutes');

app.use('/api/tasks', taskRoutes);
```

Giải thích:

- `express.Router()`: Tạo một mini-app độc lập với đầy đủ khả năng định nghĩa route và middleware.
- Đường dẫn `'/'` ở router là tương đối — khi gắn vào `app.use('/api/tasks', ...)` nó trở thành `/api/tasks/`.
- Mỗi nhóm route (tasks, auth, ai) nằm trong file riêng, dễ quản lý khi ứng dụng lớn lên.

2.5. MySQL

2.5.1. Giới thiệu

MySQL là hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở do Oracle Corporation phát triển, sử dụng ngôn ngữ SQL để truy vấn và thao tác dữ liệu. MySQL hoạt động theo mô hình Client-Server: server MySQL lưu trữ và quản lý dữ liệu, các ứng dụng (client) kết nối thông qua giao thức mạng để thực hiện các thao tác.

2.5.2. Các khái niệm cốt lõi

a) Bảng (Table) và Cột (Column)

Dữ liệu trong MySQL được tổ chức thành các bảng, mỗi bảng gồm nhiều cột và hàng, mỗi cột có kiểu dữ liệu xác định.

```
CREATE TABLE users (  
  id      INT AUTO_INCREMENT PRIMARY KEY,  
  username VARCHAR(255) NOT NULL,  
  email   VARCHAR(255) NOT NULL UNIQUE,  
  password VARCHAR(255) NULL,  
  display_name VARCHAR(255),  
  google_id VARCHAR(255) UNIQUE,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Giải thích:

- INT AUTO_INCREMENT PRIMARY KEY: Số nguyên, tự tăng 1 sau mỗi bản ghi, làm khóa chính (định danh duy nhất).
- VARCHAR(255): Chuỗi ký tự có độ dài thay đổi, tối đa 255 ký tự.
- NOT NULL: Ràng buộc bắt buộc — cột này không được để trống.
- UNIQUE: Ràng buộc duy nhất — không có 2 hàng nào có cùng giá trị ở cột này.
- DEFAULT CURRENT_TIMESTAMP: Tự động ghi thời gian hiện tại khi chèn bản ghi.

b) Truy vấn cơ bản (CRUD)

-- SELECT: Đọc dữ liệu

```
SELECT * FROM tasks WHERE user_id = 1 ORDER BY due_date ASC;
```

-- INSERT: Thêm dữ liệu mới

```
INSERT INTO tasks (user_id, title, priority, due_date, status) VALUES (1, 'Nộp báo cáo', '1', '2026-05-30 17:00:00', 'todo');
```

-- UPDATE: Cập nhật dữ liệu

```
UPDATE tasks SET status = 'done' WHERE id = 5 AND user_id = 1;
```

-- DELETE: Xóa dữ liệu

```
DELETE FROM tasks WHERE id = 5 AND user_id = 1;
```

c) Prepared Statements (Câu lệnh chuẩn bị)

Toàn bộ câu truy vấn trong FocusFlow đều sử dụng Prepared Statements với dấu ? làm placeholder, giúp ngăn chặn triệt để lỗ hổng bảo mật SQL Injection.

```
const [tasks] = await pool.execute(  
  'SELECT * FROM tasks WHERE user_id = ? AND status != ?',  
  [userId, 'done']  
);
```

Giá trị được truyền riêng trong mảng tham số; MySQL tự động escape ký tự đặc biệt, không cho phép chèn lệnh SQL độc hại vào truy vấn.

2.6. RESTful API

2.6.1. Giới thiệu

REST (Representational State Transfer) là phong cách kiến trúc cho các hệ thống phân tán. RESTful API là API được thiết kế tuân theo nguyên tắc REST, sử dụng

Website Lập Lịch và Quản Lý Công Việc

dùng các phương thức HTTP chuẩn (GET, POST, PUT, DELETE) để thao tác với tài nguyên.

2.6.2. Nguyên tắc thiết kế RESTful API

Các đường dẫn API trong FocusFlow được thiết kế theo chuẩn RESTful để đảm bảo tính tường minh và chuẩn hóa:

- GET /api/tasks — Lấy danh sách công việc (hiển thị trang Tasks).
- POST /api/tasks — Tạo công việc mới (nhấn nút "Thêm mới").
- PUT /api/tasks/:id — Cập nhật công việc (sửa tiêu đề, đổi trạng thái).
- DELETE /api/tasks/:id — Xóa công việc khỏi hệ thống.
- POST /api/auth/login — Đăng nhập (gửi email + mật khẩu).
- POST /api/auth/register — Đăng ký tài khoản mới.
- POST /api/ai/chat — Gửi tin nhắn đến trợ lý AI.

2.7. JWT (JSON Web Token)

2.7.1. Giới thiệu

JWT là tiêu chuẩn mở (RFC 7519) dùng để truyền tải thông tin xác thực giữa các bên dưới dạng JSON. JWT được sử dụng rộng rãi trong xác thực (authentication) cho API RESTful, thay thế phương pháp session truyền thống.

2.7.2. Cấu trúc JWT

Một JWT token gồm 3 phần ngăn cách bằng dấu chấm (.):

eyJhbGciOiJIUzI1NiJ9.eyJpZCI6MSwidXNlcm5hbWUiOiJ0ZXN0In0.xxx

|----- Header -----|----- Payload -----|-- Signature --|

- Header: Chứa thuật toán mã hóa (HS256) và loại token (JWT).
- Payload: Chứa dữ liệu (claims) — id người dùng, username, thời hạn.
- Signature: Chữ ký số được tạo bằng cách mã hóa Header + Payload + Secret Key, đảm bảo token không bị giả mạo.

2.8. Zustand (State Management)

2.8.1. Giới thiệu

Zustand là thư viện quản lý trạng thái (state management) nhẹ nhàng cho React, được tạo bởi nhóm Poimandres. Zustand dùng cú pháp đơn giản, không yêu cầu boilerplate phức tạp như Redux, và hoạt động bên ngoài React component tree, tránh re-render không cần thiết.

2.8.2. Cú pháp cơ bản

```
import { create } from 'zustand';

const useAuthStore = create((set) => ({
  user: null,
  token: null,
  isAuthenticated: false,
  login: async (email, password) => {
    const { token, user } = await authApi.login({ email, password });
    set({ token, user, isAuthenticated: true });
  },
  logout: () => set({ user: null, token: null, isAuthenticated: false }),
}));

function Header() {
  const user = useAuthStore((state) => state.user);
  const logout = useAuthStore((state) => state.logout);
  return <button onClick={logout}>Đăng xuất, {user?.display_name}</button>;
}
```

Giải thích:

- `create((set) => ({...}))`: Tạo store. `set` là hàm để cập nhật state.

Website Lập Lịch và Quản Lý Công Việc

- `set({ isAuthenticated: true })`: Cập nhật một phần state (shallow merge).
- `useAuthStore((state) => state.user)`: Selector — chỉ subscribe vào user, component không re-render khi token thay đổi.

2.9. TanStack Query (React Query)

2.9.1. Giới thiệu

TanStack Query (trước đây là React Query) là thư viện quản lý trạng thái server (server state) cho React. Nó giải quyết bài toán fetching, caching, synchronizing và updating dữ liệu từ API một cách tự động và hiệu quả.

2.9.2. Cú pháp cơ bản

```
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
```

```
function TaskList() {
```

```
  const queryClient = useQueryClient();
```

```
  const { data: tasks = [], isLoading } = useQuery({
```

```
    queryKey: ['tasks'],
```

```
    queryFn: () => tasksApi.list(),
```

```
  });
```

```
  const createTaskMutation = useMutation({
```

```
    mutationFn: (taskData) => tasksApi.create(taskData),
```

```
    onSuccess: () => queryClient.invalidateQueries({ queryKey: ['tasks'] }),
```

```
  });
```

```
}
```

Giải thích:

Website Lập Lịch và Quản Lý Công Việc

- `useQuery`: Hook để đọc dữ liệu. Tự động cache, retry khi lỗi, refetch khi người dùng quay lại tab.
- `queryKey`: `['tasks']`: Khóa cache duy nhất, dùng để identify và invalidate.
- `useMutation`: Hook để thay đổi dữ liệu (POST, PUT, DELETE).
- `invalidateQueries`: Đánh dấu cache là "cũ" → tự động fetch lại dữ liệu mới nhất.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Tổng quan chương 3

Trong chu trình phát triển phần mềm, giai đoạn phân tích và thiết kế hệ thống đóng vai trò quyết định, bắc cầu giữa các yêu cầu lý thuyết (đã được đề cập ở Chương 1 và Chương 2) và mã nguồn thực tế sẽ triển khai (ở Chương 4). Việc xây dựng hệ thống quản lý lịch trình tích hợp AI **FocusFlow** đòi hỏi sự kết hợp chặt chẽ giữa các tác vụ thời gian thực, đồng bộ hóa nền (background tasks), tương tác ngôn ngữ tự nhiên và bảo mật thông tin.

Trong chương này, báo cáo tập trung vào việc mô tả chi tiết bài toán dưới góc nhìn hệ thống và phân tích logic. Nội dung chương bao gồm:

1. **Phân tích bài toán:** Chỉ ra rõ ràng các vấn đề cốt lõi của người dùng, giải pháp giải quyết mang tính hệ thống, và liệt kê đầy đủ các yêu cầu chức năng cũng như phi chức năng của hệ thống.
2. **Phân tích Use Case:** Thiết lập biểu đồ Use Case tổng quát và đi sâu đặc tả chi tiết các Use Case quan trọng nhất. Mỗi Use Case không chỉ được mô tả bằng bảng mà còn đi kèm phân tích các tình huống, kịch bản nghiệp vụ có thể phát sinh trong thực tế.
3. **Biểu đồ tuần tự (Sequence Diagram):** Mô tả dòng chảy dữ liệu, trình tự tương tác giữa các tác nhân (Actors) và các thành phần bên trong hệ thống (Frontend, Middleware, Backend Controllers, Services, Database) cho từng quy trình chính. Mỗi biểu đồ đều đi kèm diễn giải kỹ thuật và giải pháp xử lý lỗi.
4. **Thiết kế cơ sở dữ liệu:** Chi tiết hóa cấu trúc dữ liệu với các bảng, kiểu dữ liệu, ràng buộc khóa chính, khóa ngoại, và biểu đồ quan hệ thực thể (ERD) hoàn chỉnh.
5. **Thiết kế kiến trúc hệ thống:** Mô hình hóa kiến trúc 3 tầng (3-tier Architecture) của ứng dụng và sơ đồ thư mục dự án thực tế, giúp làm rõ tính module hóa và khả năng mở rộng của hệ thống.

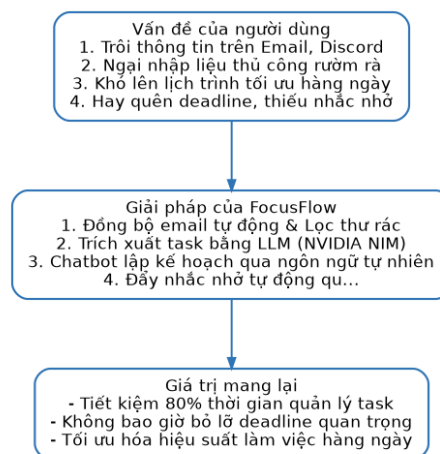
3.2. Phân tích bài toán

3.2.1. Mô tả bài toán chi tiết và logic

Trong kỷ nguyên số, người dùng cá nhân (như sinh viên, nhân viên văn phòng, freelancer) đối mặt với hai vấn đề lớn trong quản lý thời gian: **sự phân tán thông tin** và **áp lực nhập liệu thủ công**.

- **Sự phân tán thông tin:** Công việc không chỉ xuất hiện từ ý thức chủ quan (tự tạo task) mà đến từ rất nhiều kênh giao tiếp ngoại vi: email từ trường học/đối tác, tin nhắn công việc từ các nhóm Discord, Slack, Zalo. Việc phải liên tục chuyển đổi qua lại giữa các ứng dụng này để ghi chép lại việc cần làm cực kỳ tốn thời gian và dễ gây bỏ sót công việc (deadline).
- **Áp lực nhập liệu và lên lịch:** Các ứng dụng To-Do truyền thống yêu cầu người dùng phải gõ từng tiêu đề, chọn ngày giờ bằng lịch, chọn mức độ ưu tiên bằng nút click. Sự rườm rà này khiến nhiều người bỏ cuộc sau vài ngày sử dụng. Đồng thời, việc tự sắp xếp lịch trình trong ngày sao cho hợp lý cũng đòi hỏi tư duy phân tích, gây mệt mỏi tinh thần (cognitive overload).

Hệ thống **FocusFlow** ra đời để giải quyết các vấn đề trên một cách có hệ thống thông qua mô hình tự động hóa thông minh phối hợp với Trí tuệ nhân tạo (AI):



Hình 3. 1: Sơ đồ minh họa cho Mô tả bài toán chi tiết và logic

3.2.2.1. Cách thức hệ thống giải quyết vấn đề của người dùng:

1. **Giải quyết sự phân tán thông tin bằng Email Sync tự động:** Hệ thống chạy ngầm một dịch vụ (worker service) định kỳ kết nối tới hộp thư cá nhân (Gmail/Outlook) qua giao thức IMAP. Dịch vụ này không lấy toàn bộ thư mà áp dụng thuật toán lọc

Website Lập Lịch và Quản Lý Công Việc

thông minh để nhận diện các email liên quan đến công việc, bài tập. Sau đó, nội dung email được gửi đến mô hình ngôn ngữ lớn (LLM) để trích xuất ra các trường thông tin chuẩn hóa: Tiêu đề công việc, Mô tả chi tiết, Hạn hoàn thành (Due Date) và Mức độ ưu tiên.

2. Giải quyết áp lực nhập liệu bằng Trợ lý AI và Cơ chế Lập kế hoạch Thông minh:

Thay vì tạo task thủ công, người dùng có thể chat trực tiếp với Trợ lý AI bằng ngôn ngữ tự nhiên (Ví dụ: "Tôi có bài tập lớn Web cần nộp vào thứ Hai tuần sau và một buổi họp nhóm lúc 2h chiều mai, hãy lên lịch giúp tôi"). AI sẽ phân tích câu nói, hiểu ngữ cảnh thời gian thực và trả về một cấu trúc kế hoạch được định dạng sẵn. Giao diện frontend sẽ render cấu trúc này thành các thẻ kế hoạch trực quan, người dùng chỉ cần nhấn "Chấp nhận" để hệ thống tự động lưu loạt công việc đó vào cơ sở dữ liệu.

3. Giải quyết vấn đề quên deadline bằng Nhắc nhở Chủ động (Proactive Notification):

Hệ thống tích hợp Discord Webhook. Định kỳ mỗi phút, hệ thống quét cơ sở dữ liệu để tìm các task sắp đến hạn trong vòng 30 phút tiếp theo mà chưa được hoàn thành. Một tin nhắn định dạng Markdown sinh động sẽ được tự động bắn thẳng vào kênh Discord cá nhân của người dùng, giúp họ nhận thức ngay lập tức mà không cần phải mở ứng dụng FocusFlow.

4. Việt hóa và Tối ưu hóa giao diện lịch:

Hệ thống cung cấp một Lịch tháng tích hợp Lịch Âm (Lunar Calendar) và hiển thị các ngày lễ truyền thống của Việt Nam. Đây là tính năng đặc biệt quan trọng với người dùng Việt Nam khi cần sắp xếp công việc cá nhân trùng với các dịp lễ hội truyền thống hoặc ngày nghỉ gia đình.

3.2.2.2. Đối tượng sử dụng hệ thống

Dưới đây là bảng phân tích cụ thể các nhóm đối tượng mục tiêu của FocusFlow, chỉ rõ nhu cầu thực tế và cách hệ thống đáp ứng tương ứng:

Bảng 3. 1: Bảng nội dung trong mục Đối tượng sử dụng hệ thống

Nhóm đối tượng	Đặc điểm hoạt động	Nhu cầu quản lý thời gian	Cách FocusFlow giải quyết
Sinh viên	- Học tập theo thời khóa biểu thay đổi. - Nhận nhiều thông báo bài tập lớn, lịch thi	- Đồng bộ nhanh bài tập từ email giảng viên. - Nhắc nhở hạn nộp	- Background worker tự quét email trường, trích xuất deadline nộp bài.

	qua email trường học (.edu.vn). - Hoạt động nhóm nhiều.	bài trước khi công nộp bài đóng. - Lên lịch ôn thi khoa học.	- Discord gửi tin nhắn nhắc nhở trước 30 phút. - Trợ lý AI phân chia lộ trình ôn thi thành các task nhỏ hàng ngày.
Nhân viên văn phòng	- Nhận hàng chục email công việc mỗi ngày từ đối tác, sếp. - Lịch họp dày đặc. - Sử dụng các công cụ chat nhóm như Discord, Slack làm việc.	- Lọc nhanh việc quan trọng từ email. - Tổng hợp lịch họp và các đầu việc cần giải quyết trong ngày. - Giao diện trực quan để cập nhật nhanh trạng thái.	- Đồng bộ email công việc, bỏ qua email quảng cáo/spam. - Dashboard tổng hợp toàn bộ lịch họp dạng timeline. - Tính năng "Quick Add" giúp thêm nhanh task phát sinh khi đang họp.
Freelancer (Làm việc tự do)	- Quản lý đồng thời nhiều dự án cho nhiều khách hàng. - Tự sắp xếp thời gian biểu hoàn toàn. - Cần đo lường hiệu suất làm việc để tối ưu hóa thu nhập.	- Phân loại công việc theo dự án/nguồn khách hàng. - Báo cáo phân tích hiệu suất làm việc để tự điều chỉnh. - Tránh bị quá tải (burnout) do nhận quá nhiều việc.	- Phân loại task theo thể nguồn (Source) và danh mục (Category). - Trang Analytics với biểu đồ SVG phân tích tỷ lệ hoàn thành công việc. - Trợ lý AI đưa ra lời khuyên phân bổ lịch trình dựa trên năng suất thực tế.
Người dùng cá nhân	- Quản lý các công việc thường nhật (dọn dẹp, mua sắm, tập thể thao, hiếu hi). - Cần sự đơn giản, không rườm rà.	- Giao diện thân thiện, dễ dùng trên cả điện thoại và máy tính. - Hỗ trợ xem lịch Âm để theo dõi các sự kiện gia đình truyền thống.	- Giao diện Glassmorphism hiện đại, dễ thao tác. - Lịch tháng tích hợp Lịch Âm - Dương song hành. - Chatbot AI hỗ trợ lập kế hoạch sinh hoạt hàng ngày thông qua trò chuyện.

3.2.2. Yêu cầu chức năng chi tiết

Hệ thống được module hóa thành 6 nhóm chức năng chính. Mỗi chức năng được gán mã Use Case (UC) để phục vụ cho quá trình thiết kế và kiểm thử:

Nhóm chức năng 1: Quản lý tài khoản (Authentication & Authorization)

- **UC01: Đăng ký tài khoản:** Cho phép người dùng mới tạo tài khoản bằng Email cá nhân. Hệ thống bắt buộc validate định dạng email, độ dài mật khẩu (tối thiểu 6 ký

Website Lập Lịch và Quản Lý Công Việc

tự). Mật khẩu được băm bằng thuật toán `bcrypt` (salt rounds = 10) trước khi lưu vào database nhằm đảm bảo an toàn tuyệt đối.

- **UC02: Đăng nhập bằng tài khoản hệ thống:** Xác thực người dùng bằng Email và Mật khẩu. Hệ thống kiểm tra thông tin đối chiếu trong cơ sở dữ liệu. Nếu khớp, sinh ra mã JWT (JSON Web Token) chứa thông tin định danh của user, ký bằng khóa bí mật (secret key) với thời hạn hết hạn là 7 ngày. Token này sẽ được lưu ở `localStorage` của trình duyệt để tự động đính kèm vào header `Authorization: Bearer` trong các API request tiếp theo.
- **UC03: Đăng nhập bằng Google OAuth 2.0:** Tích hợp SDK đăng nhập của Google. Hệ thống nhận `credential` (ID Token) từ Google, gửi lên backend để xác thực tính hợp lệ với Google API. Nếu người dùng đăng nhập lần đầu, hệ thống tự động tạo một bản ghi user mới trong bảng `users` với trường `google_id`, lấy ảnh đại diện (`avatar_url`) và tên hiển thị trực tiếp từ tài khoản Google.
- **UC04: Đăng xuất:** Xóa bỏ JWT Token khỏi `localStorage` và đặt lại trạng thái xác thực trong Zustand store về mặc định (`isAuthenticated = false`, `user = null`), sau đó chuyển hướng người dùng về trang đăng nhập nhằm ngăn chặn truy cập trái phép.

Nhóm chức năng 2: Quản lý công việc (Task Management)

- **UC05: Xem danh sách công việc đa góc nhìn:** Hiển thị danh sách công việc của người dùng. Hệ thống hỗ trợ chia thành các tab thời gian: "Hôm nay", "Sắp tới" (các việc có deadline sau hôm nay), "Tất cả" (toàn bộ việc đang có), và "Đã hoàn thành". Hỗ trợ tìm kiếm thời gian thực theo tiêu đề.
- **UC06: Tạo công việc mới đầy đủ thuộc tính:** Tạo công việc thông qua form chi tiết bao gồm: tiêu đề (bắt buộc), mô tả chi tiết (tùy chọn), mức độ ưu tiên (1 - Cao, 2 - Trung bình, 3 - Thấp), hạn hoàn thành (ngày và giờ cụ thể), danh mục phân loại. Nguồn tạo được gắn nhãn mặc định là "Custom".
- **UC07: Thêm nhanh công việc (Quick Add):** Một ô nhập liệu luôn xuất hiện ở đầu trang danh sách công việc. Người dùng chỉ cần gõ tiêu đề và nhấn Enter, hệ thống lập tức tạo công việc với các giá trị mặc định: deadline là ngày hôm nay, mức độ ưu tiên trung bình. Điều này giảm thiểu tối đa rào cản tương tác.

Website Lập Lịch và Quản Lý Công Việc

- **UC08: Cập nhật công việc linh hoạt:** Hỗ trợ sửa nhanh tiêu đề ngay trên danh sách (inline edit), thay đổi mức độ ưu tiên, cập nhật deadline, và đánh dấu hoàn thành nhanh bằng cách click chọn checkbox. Trạng thái task sẽ chuyển đổi linh hoạt giữa `todo` và `done`.
- **UC09: Xóa công việc:** Xóa vĩnh viễn công việc khỏi cơ sở dữ liệu kèm theo thông báo xác nhận để tránh người dùng click nhầm.
- **UC10: Lọc công việc theo ngày:** Bộ chọn ngày trực quan cho phép người dùng xem danh sách công việc của một ngày bất kỳ trong quá khứ hoặc tương lai.
- **UC11: Phân loại và lọc theo nguồn (Source Filtering):** Lọc danh sách công việc dựa trên nguồn gốc sinh ra công việc đó: tạo thủ công (`Custom`), trích xuất từ email (`Email`), chat với AI (`AI`), hay từ các nguồn mở rộng khác. Hệ thống hiển thị số lượng công việc tương ứng bên cạnh mỗi nguồn để người dùng dễ theo dõi.

Nhóm chức năng 3: Dashboard và Lịch trình (Dashboard & Scheduling)

- **UC12: Dashboard tổng quan thông minh:** Trang chủ hiển thị chào mừng người dùng cá nhân hóa theo thời gian trong ngày (Sáng/Chiều/Tối). Cung cấp các thẻ thống kê nhanh dạng số lượng: Việc cần làm hôm nay, việc quá hạn, việc hoàn thành. Đồng thời hiển thị danh sách công việc quan trọng nhất cần giải quyết ngay.
- **UC13: Lịch trình chi tiết trong ngày (Today's Timeline):** Hiển thị công việc của ngày hôm nay sắp xếp theo tuyến tính thời gian. Cho phép người dùng đổi thứ tự ưu tiên thực hiện bằng cách di chuyển vị trí, đánh dấu hoàn thành hoặc xóa nhanh trực tiếp trên timeline.
- **UC14: Xem bảng lịch trình mở rộng:** Một modal hiển thị danh sách công việc dưới dạng bảng chi tiết đầy đủ thông tin: Tên công việc, Mức độ ưu tiên (có màu sắc nhận diện), Thời gian thực hiện, Thời gian còn lại đến deadline, Nguồn tạo, và các nút thao tác nhanh.
- **UC15: Lịch tháng đa chức năng (Interactive Monthly Calendar):** Giao diện lưới lịch tháng trực quan. Mỗi ô ngày hiển thị tối đa 3 công việc tiêu biểu của ngày đó. Đặc biệt tích hợp hiển thị ngày Âm lịch (chữ nhỏ màu xám bên dưới ngày Dương lịch) và bôi đỏ/hiển thị tên các ngày lễ lớn của Việt Nam (Ví dụ: Tết Nguyên Đán, Giỗ tổ Hùng Vương, Quốc khánh 2/9, v.v.).

Nhóm chức năng 4: Trợ lý AI thông minh (AI Assistant)

- **UC16: Chat tương tác tự nhiên với AI:** Giao diện chat dạng luồng hội thoại. Trợ lý AI được cung cấp System Prompt định hình cá tính thông minh, hiểu được thời gian hiện tại của hệ thống và danh sách toàn bộ công việc chưa hoàn thành của người dùng để trả lời sát thực tế nhất.
- **UC17: AI tự động đề xuất lịch trình:** Khi người dùng yêu cầu lên kế hoạch, AI phân tích dữ liệu đầu vào và trả về một khối định dạng XML `[{"title": "...", "due_date": "...", "priority": "..."}]`. Hệ thống tự động bắt và parse thẻ XML này để hiển thị một "Plan Card" trực quan trên khung chat, cho phép người dùng xem trước, chỉnh sửa và nhấn nút lưu hàng loạt task vào DB.
- **UC18: AI truy vấn và giải đáp email:** Khi người dùng hỏi về email mới (Ví dụ: "Tôi có email nào quan trọng gần đây không?"), backend của AI Assistant sẽ kết nối hòm thư IMAP lấy nhanh 5 email gần nhất, gửi kèm vào Prompt để AI tóm tắt và trả lời trực tiếp cho người dùng trong cửa sổ chat.
- **UC19: Đồng bộ email tự động nền (Background Email Sync):** Background worker chạy định kỳ mỗi 5 phút. Quét các mail chưa đọc, sử dụng bộ lọc regex để lọc thư rác. Gửi email hợp lệ qua NVIDIA NIM AI để phân tích thông tin công việc, tự động ghi nhận vào database bảng `tasks` với trường `email_uid` độc bản và tạo một bản ghi thông báo hệ thống.

Nhóm chức năng 5: Thông báo (Notification Services)

- **UC20: Quản lý thông báo trong ứng dụng (In-app Notification):** Khi có task mới được tạo tự động từ email, hệ thống sinh ra thông báo trong app. Biểu tượng chuông thông báo hiển thị số lượng thông báo chưa đọc. Người dùng có thể click xem chi tiết hoặc nhấn "Đánh dấu tất cả đã đọc".
- **UC21: Nhắc nhở chủ động qua Discord Webhook:** Hệ thống tích hợp tự động với Discord. Background worker mỗi phút quét tìm các task chưa hoàn thành có hạn deadline nằm trong khoảng 30 phút tới. Hệ thống định dạng tin nhắn Markdown đẹp mắt (có emoji, bảng thông tin công việc) và gửi POST request đến Discord Webhook URL được cấu hình bởi người dùng. Sau khi gửi thành công, cập nhật cờ `discord_notified = 1` để tránh gửi lặp.

Nhóm chức năng 6: Phân tích hiệu suất công việc (Productivity Analytics)

- **UC22: Biểu đồ cột hiệu suất tuần:** Sử dụng SVG thuần để vẽ biểu đồ trực quan hóa số lượng công việc hoàn thành trong 7 ngày gần nhất. Có hiệu ứng hover hiển thị chi tiết số lượng task cụ thể của từng ngày.
- **UC23: Thống kê hiệu suất tổng quát:** Tính toán tỷ lệ hoàn thành công việc theo chu kỳ tuần/tháng, chỉ ra ngày làm việc năng suất nhất trong tuần (hoàn thành nhiều task nhất) và đưa ra gợi ý thông minh dựa trên dữ liệu hiệu suất của người dùng.
- **UC24: Biểu đồ phân bổ công việc theo nguồn:** Biểu đồ thanh ngang so sánh số lượng công việc đến từ các nguồn khác nhau (Custom, Email, AI, Discord, v.v.), giúp người dùng nhận biết nguồn công việc chính của mình.
- **UC25: Biểu đồ phân bổ công việc theo mức độ ưu tiên:** Hiển thị tỉ lệ công việc theo mức độ ưu tiên (Cao / Trung bình / Thấp) để người dùng đánh giá mức độ quan trọng của các đầu việc mình đang xử lý.

Nhóm chức năng 7: Workspace và cộng tác nhóm (Workspace & Team Collaboration)

- **UC26: Tạo và quản lý Workspace:** Cho phép người dùng tạo không gian làm việc chung cho từng nhóm học tập, dự án hoặc phòng ban; cập nhật tên, mô tả, màu nhận diện và trạng thái hoạt động của Workspace.
- **UC27: Quản lý thành viên và phân quyền:** Chủ sở hữu Workspace có thể mời thành viên qua email, cấp vai trò Owner/Admin/Member, xóa thành viên và kiểm soát quyền thao tác trên công việc nhóm.
- **UC28: Tạo lịch và quản lý lịch nhóm:** Cho phép tạo lịch làm việc chung, hiển thị các công việc/sự kiện của nhóm theo ngày, theo thành viên được giao và theo tiến độ thực hiện.
- **UC29: Giao việc và theo dõi tiến độ nhóm:** Chủ sở hữu hoặc Admin có thể giao việc cho từng thành viên, theo dõi trạng thái todo/done, xem tiến độ chung của nhóm và cập nhật công việc trực tiếp trên bảng Workspace.

3.2.3. Yêu cầu phi chức năng (Non-Functional Requirements)

Bảng 3. 2: Bảng nội dung trong mục Yêu cầu phi chức năng (Non-Functional Requirements)

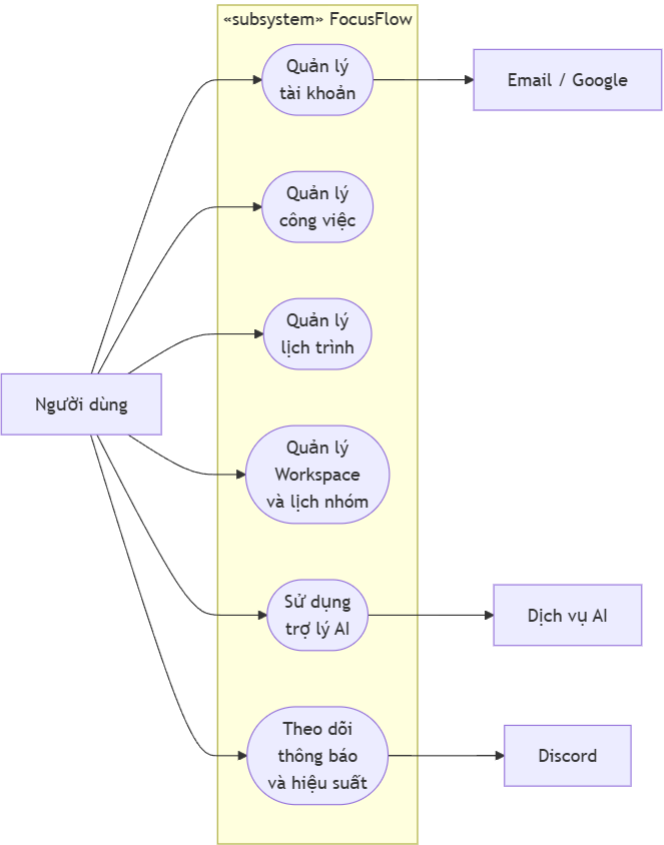
Tiêu chuẩn đánh giá	Yêu cầu kỹ thuật chi tiết	Giải pháp thiết kế tương ứng
Hiệu năng và Tốc độ	- Thời gian phản hồi giao diện người dùng (UI Response Time) phải nhỏ hơn 200ms cho các tác vụ thông thường (Click tab, mở modal). - API backend phản hồi trong dưới 500ms. - Tốc độ tải trang đầu tiên dưới 2 giây.	- Sử dụng ReactJS SPA kết hợp với state local cực kỳ nhanh. - Áp dụng TanStack Query để cache dữ liệu API phía client, hạn chế tối đa các request trùng lặp lên database. - Sử dụng thư viện mysql2 với kết nối dạng Connection Pool để tái sử dụng kết nối database nhanh chóng.
Bảo mật thông tin	- Mật khẩu người dùng bắt buộc phải được mã hóa trước khi lưu trữ. - Chống các lỗ hổng bảo mật phổ biến như SQL Injection, XSS. - Xác thực phiên làm việc an toàn.	- Sử dụng thuật toán băm một chiều bcrypt để mã hóa mật khẩu. - Áp dụng Prepared Statements (sử dụng dấu ? trong câu truy vấn SQL) để loại bỏ hoàn toàn nguy cơ SQL Injection. - Xác thực bằng cơ chế JWT token ký số bảo mật, truyền qua header Authorization thay vì lưu cookie trực tiếp để tránh lỗi CSRF.
Độ tin cậy và Khả dụng	- Hệ thống hoạt động liên tục 24/7. - Các dịch vụ chạy nền (email sync, Discord reminder) không được làm treo hay chậm luồng xử lý HTTP chính của backend Express.js.	- Tách biệt luồng xử lý HTTP của Express.js và luồng xử lý background worker. - Sử dụng cơ chế bắt lỗi try-catch chặt chẽ trong các vòng lặp setInterval của background worker, đảm bảo nếu một lần quét bị lỗi kết nối mạng thì ứng dụng vẫn tiếp tục chạy và quét lại ở chu kỳ tiếp theo mà không bị crash.
Khả năng bảo trì và Mở rộng	- Dễ dàng tích hợp thêm các kênh liên lạc mới trong tương lai (Ví dụ: đồng bộ tin nhắn Zalo, gửi thông báo qua Telegram). - Mã nguồn sạch, dễ đọc và bảo trì.	- Cấu trúc code backend theo mô hình MVC tách biệt (Routes -> Middleware -> Controller -> Database Access). - Module hóa các dịch vụ tích hợp bên ngoài (NVIDIA NIM API, IMAP, Discord Webhook) thành các service riêng biệt, dễ dàng thay thế hoặc mở rộng.
Trải nghiệm người dùng (UX)	- Giao diện bắt mắt, trực quan, hỗ trợ tiếng Việt hoàn toàn. - Thiết kế Responsive hoạt	- Áp dụng phong cách thiết kế Glassmorphism hiện đại với các màu sắc HSL phối hợp hài

	động mượt mà trên cả máy tính (Desktop) và điện thoại di động (Mobile).	hòa, độ tương phản tốt. - Sử dụng CSS Media Queries để tối ưu giao diện cho các kích thước màn hình khác nhau. - Thêm các micro-animations và Toast Notifications (react-hot-toast) để tương tác trực quan với người dùng.
--	---	--

3.3. Phân tích Use Case

3.3.1. Biểu đồ Use Case tổng quan

Biểu đồ Use Case mô tả trực quan các ranh giới của hệ thống **FocusFlow**, các tác nhân bên ngoài (Actors) và mối quan hệ tương tác của họ với 25 chức năng cốt lõi của hệ thống.



Hình 3. 2: Sơ đồ minh họa cho Biểu đồ Use Case tổng quan

Phân tích vai trò của các Tác nhân (Actors) trong hệ thống:

1. **Người dùng (User - Chủ thể chính):** Là tác nhân trực tiếp tương tác với giao diện Frontend. Họ thực hiện các thao tác quản trị tài khoản cá nhân, xem và chỉnh sửa

Website Lập Lịch và Quản Lý Công Việc

danh sách công việc, quản lý lịch biểu Dương-Âm, trò chuyện trực tiếp với trợ lý AI và theo dõi các biểu đồ phân tích năng suất cá nhân.

2. **Hệ thống tự động (Automation Service - Tác nhân chạy ngầm):** Hoạt động hoàn toàn độc lập ở phía server (background process). Tác nhân này thực hiện các công việc định kỳ theo cơ chế cron timer: quét hòm thư email, gửi tin nhắn cảnh báo đến Discord của người dùng.
3. **AI Service (NVIDIA NIM - Tác nhân dịch vụ):** Cung cấp API trí tuệ nhân tạo (LLM). Tác nhân này nhận các prompt văn bản từ backend FocusFlow để thực hiện hai nhiệm vụ: một là trả lời chatbot tương tác của người dùng, hai là phân tích cấu trúc email thô để trích xuất thông tin task.
4. **Email Server (Gmail/Outlook IMAP - Tác nhân dữ liệu ngoại vi):** Đóng vai trò là nguồn cấp thông tin công việc đầu vào của người dùng. Hệ thống kết nối tới đây để lấy các email mới nhất chưa đọc.
5. **Discord (Discord Webhook API - Tác nhân thông báo):** Nhận dữ liệu payload dưới dạng Markdown từ backend FocusFlow để hiển thị tin nhắn cảnh báo đến người dùng thông qua kênh chat nhóm Discord.

3.3.2. Đặc tả chi tiết các Use Case quan trọng

Để làm rõ logic nghiệp vụ và các tình huống phát sinh thực tế, phần này đặc tả chi tiết 6 Use Case cốt lõi nhất của hệ thống FocusFlow, bao gồm cả luồng xử lý chính và các luồng ngoại lệ/lỗi có thể xảy ra.

3.3.2.1. Use Case: Đăng nhập hệ thống

- **Mục đích:** Xác thực danh tính của người dùng để cấp quyền truy cập vào các tính năng cá nhân hóa trên hệ thống FocusFlow. Đảm bảo an toàn thông tin dữ liệu của từng cá nhân.
- **Tình huống thực tế có thể xảy ra:**
 - Người dùng nhập sai mật khẩu hoặc email chưa từng đăng ký.
 - Người dùng cố tình thực hiện tấn công brute-force gửi liên tiếp yêu cầu đăng nhập.
 - Token hết hạn khi người dùng đang sử dụng ứng dụng.

Bảng 3. 3: Bảng nội dung trong mục Use Case: Đăng nhập hệ thống

Thuộc tính	Chi tiết đặc tả
Tên Use Case	Đăng nhập hệ thống
Actor	Người dùng (chưa xác thực)
Mô tả ngắn	Người dùng cung cấp email và mật khẩu thông qua form đăng nhập để truy cập vào hệ thống.
Tiền điều kiện	Người dùng đã đăng ký tài khoản thành công trên hệ thống và tài khoản đang ở trạng thái hoạt động.
Hậu điều kiện	Người dùng được xác thực, hệ thống lưu JWT Token vào bộ nhớ cục bộ (localStorage), trạng thái xác thực trong Zustand Store được cập nhật và chuyển hướng người dùng tới Dashboard.
Luồng chính (Basic Flow)	1. Người dùng truy cập trang Đăng nhập. 2. Người dùng nhập địa chỉ Email và Mật khẩu vào form. 3. Người dùng click vào nút "Đăng nhập". 4. Hệ thống thực hiện kiểm tra dữ liệu đầu vào phía client (validate email đúng định dạng, mật khẩu không để trống). 5. Frontend gửi request POST /api/auth/login chứa dữ liệu đăng nhập tới Backend. 6. Backend thực hiện câu lệnh truy vấn tìm kiếm người dùng trong bảng users theo email. 7. Backend so sánh mật khẩu người dùng nhập vào với mật khẩu đã được băm lưu trong DB bằng bcrypt.compare(). 8. Nếu trùng khớp, Backend tạo một mã JWT Token được ký bằng secret key của hệ thống. 9. Backend phản hồi về Frontend mã JWT Token cùng thông tin cơ bản của user (id, email, display_name, avatar_url) dưới dạng JSON với mã trạng thái HTTP 200 OK. 10. Frontend nhận phản hồi, lưu JWT và thông tin user vào localStorage, cập nhật state toàn cục của authStore. 11. Hệ thống hiển thị thông báo chào mừng thành công và chuyển hướng người dùng đến trang Dashboard (/dashboard).
Luồng ngoại lệ (Exception Flows)	Ngoại lệ 1: Định dạng email không hợp lệ hoặc để trống - Tại bước 4, nếu dữ liệu không đạt yêu cầu, Frontend lập tức chặn không gửi request và hiển thị viền đỏ báo lỗi tại ô nhập liệu tương ứng. Ngoại lệ 2: Email không tồn tại trên hệ thống - Tại bước 6, Backend truy vấn bảng users trả về kết quả rỗng. - Backend dừng xử lý và trả về lỗi HTTP 400

	Bad Request kèm thông báo chung: "Email hoặc mật khẩu không chính xác". (Thiết kế này nhằm bảo mật, tránh kẻ tấn công dò tìm danh sách email tồn tại). - Frontend hiển thị thông báo lỗi (Toast error) để người dùng thử lại. Ngoại lệ 3: Mật khẩu không chính xác - Tại bước 7, hàm so sánh mật khẩu trả về giá trị false. - Backend lập tức hủy xử lý, trả về lỗi HTTP 400 Bad Request kèm thông báo: "Email hoặc mật khẩu không chính xác".
--	---

3.3.2.2. Use Case: Tạo công việc mới

- **Mục đích:** Cho phép người dùng chủ động lưu lại các đầu việc cần làm của họ vào cơ sở dữ liệu để lên kế hoạch quản lý thời gian.
- **Tình huống thực tế có thể xảy ra:**
 - Người dùng để trống tiêu đề công việc (chỉ gõ mô tả).
 - Người dùng chọn hạn hoàn thành (Due Date) trong quá khứ so với thời điểm hiện tại.
 - Phiên làm việc của người dùng đã hết hạn (JWT Token hết hạn) trong lúc họ đang soạn thảo công việc.

Bảng 3. 4: Bảng nội dung trong mục Use Case: Tạo công việc mới

Thuộc tính	Chi tiết đặc tả
Tên Use Case	Tạo công việc mới
Actor	Người dùng (đã xác thực)
Mô tả ngắn	Người dùng điền các thông tin của một công việc mới vào form và lưu lại trên hệ thống.
Tiền điều kiện	Người dùng đã đăng nhập thành công và có JWT Token hợp lệ lưu tại client.
Hậu điều kiện	Công việc mới được thêm vào bảng tasks trong database gắn với user_id của tác nhân, danh sách công việc hiển thị trên màn hình được cập nhật tự động.
Luồng chính (Basic Flow)	1. Người dùng nhấp vào nút "Thêm mới" trên trang quản lý công việc hoặc nút "+" trên thanh tiện ích. 2. Hệ thống hiển thị cửa sổ Modal chứa Form tạo công việc. 3. Người dùng nhập các trường: Tiêu đề công việc, Mô tả chi tiết, chọn Mức độ ưu tiên (Cao/Trung bình/Thấp), chọn Ngày và giờ deadline (Due Date), chọn Danh mục phân loại.

	4. Người dùng nhấp vào nút "Lưu". 5. Frontend thực hiện gửi request POST /api/tasks đến Backend kèm JWT Token đính kèm ở header Authorization. 6. Hệ thống chạy qua Middleware xác thực. Middleware giải mã JWT Token thành công và gán ID của người dùng vào đối tượng request (req.user.id). 7. Backend kiểm tra tính hợp lệ của dữ liệu (Tiêu đề bắt buộc, độ dài phù hợp). 8. Backend thực hiện câu lệnh INSERT INTO tasks để lưu công việc mới vào database. 9. Database thực hiện lưu thành công và trả về mã ID của bản ghi vừa tạo. 10. Backend phản hồi HTTP 201 Created kèm theo dữ liệu đầy đủ của task vừa tạo. 11. Frontend sử dụng TanStack Query để kích hoạt cơ chế làm mới cache (invalidateQueries(['tasks'])). Giao diện tự động gửi request GET nhằm để cập nhật danh sách công việc mới nhất. 12. Hệ thống đóng Modal và hiển thị thông báo: "Đã thêm công việc thành công".
Luồng ngoại lệ (Exception Flows)	Ngoại lệ 1: Tiêu đề công việc bị bỏ trống - Tại bước 7, Backend phát hiện trường title bị rỗng. - Backend trả về lỗi HTTP 400 Bad Request kèm thông điệp: "Tiêu đề công việc là bắt buộc". Giao diện hiển thị lỗi ngay dưới ô nhập tiêu đề để người dùng bổ sung. Ngoại lệ 2: Hạn hoàn thành ở quá khứ - Tại bước 7, Backend phát hiện ngày deadline được chọn nhỏ hơn thời gian hiện tại của Server. - Hệ thống vẫn chấp nhận tạo nhưng gắn cờ cảnh báo quá hạn lập tức trên UI để người dùng nhận biết. Ngoại lệ 3: Phiên làm việc hết hạn (Token invalid/expired) - Tại bước 6, Middleware giải mã JWT thất bại (lỗi signature hoặc expired). - Middleware trả về lỗi HTTP 401 Unauthorized. - Frontend nhận mã 401, lập tức xóa thông tin đăng nhập trong store, hiển thị thông báo: "Phiên làm việc đã hết hạn, vui lòng đăng nhập lại" và chuyển hướng người dùng về trang login.

3.3.2.3. Use Case: Chat tương tác với trợ lý AI

- **Mục đích:** Cung cấp trải nghiệm quản lý công việc rảnh tay bằng cách cho phép người dùng ra lệnh hoặc hỏi đáp trực tiếp bằng ngôn ngữ tự nhiên để AI tổng hợp thông tin và lên kế hoạch.
- **Tình huống thực tế có thể xảy ra:**
 - Người dùng hỏi một câu hỏi chung chung không liên quan đến công việc (ví dụ: thời tiết, lập trình, giải toán).
 - Người dùng ra lệnh lên kế hoạch cho ngày mai, AI sinh ra kế hoạch nhưng người dùng muốn chỉnh sửa một số task trước khi đồng ý lưu.
 - Kết nối đến dịch vụ AI (NVIDIA NIM API) bị quá tải hoặc mất kết nối mạng.

***Bảng 3. 5:** Bảng nội dung trong mục Use Case: Chat tương tác với trợ lý AI*

Thuộc tính	Chi tiết đặc tả
Tên Use Case	Chat tương tác với trợ lý AI
Actor	Người dùng (đã xác thực), AI Service (NVIDIA NIM)
Mô tả ngắn	Người dùng trò chuyện với Trợ lý ảo FocusFlow bằng tiếng Việt để quản lý công việc và lên lịch trình thông qua ngôn ngữ tự nhiên.
Tiền điều kiện	Người dùng đã đăng nhập, hệ thống đã được cấu hình API Key của NVIDIA NIM trên Backend.
Hậu điều kiện	Người dùng nhận được phản hồi phù hợp từ AI. Nếu câu trả lời chứa kế hoạch công việc, một Plan Card sẽ hiển thị để người dùng phê duyệt.
Luồng chính (Basic Flow)	1. Người dùng truy cập trang Trợ lý AI (AI Assistant). 2. Người dùng nhập nội dung câu hỏi/yêu cầu vào ô chat (Ví dụ: "Lên lịch cho tôi học ReactJS từ 8h đến 10h sáng mai và đi siêu thị lúc 17h"). 3. Người dùng nhấn nút "Gửi" hoặc phím Enter. 4. Frontend hiển thị tin nhắn của người dùng lên khung chat, hiển thị trạng thái "AI đang suy nghĩ...". 5. Frontend gửi request POST /api/ai/chat chứa nội dung tin nhắn và lịch sử hội thoại lên Backend. 6. Backend tiếp nhận request, thực hiện lấy danh sách các công việc chưa hoàn thành hiện có của user từ DB để làm thông tin ngữ cảnh (Context). 7. Backend kiểm tra nội dung tin nhắn có chứa từ khóa liên quan đến Email (thư, email, mail). Nếu có, hệ thống kết nối IMAP để lấy 5 email công việc gần nhất.

	8. Backend xây dựng một System Prompt chi tiết chứa: vai trò trợ lý FocusFlow, thời gian thực hiện tại, danh sách task hiện có của user, dữ liệu email (nếu có), và quy tắc bắt buộc nếu có kế hoạch thì phải đóng gói dưới thẻ [JSON]. 9. Backend gửi yêu cầu đến NVIDIA NIM API sử dụng model openai/gpt-oss-120b. 10. Dịch vụ AI trả về kết quả là đoạn hội thoại tiếng Việt có kèm khối dữ liệu XML. 11. Backend nhận kết quả và chuyển tiếp phản hồi về Frontend dạng JSON. 12. Frontend thực hiện phân tách (parse) chuỗi văn bản nhận được. Phát hiện thẻ và tách phần JSON kế hoạch ra khỏi đoạn chat thường. 13. Frontend hiển thị câu trả lời của AI đồng thời render một component "Plan Card" trực quan liệt kê danh sách các task đề xuất kèm theo checkbox và nút "Chấp nhận kế hoạch". 14. Người dùng kiểm tra, có thể tích chọn/bỏ chọn một số task, sau đó nhấn "Chấp nhận kế hoạch". 15. Frontend gửi request POST /api/ai/plan/commit chứa mảng các task được duyệt lên Backend. 16. Backend thực hiện chèn đồng thời (batch insert) các task này vào database, sau đó trả về kết quả thành công. 17. Frontend cập nhật lại danh sách công việc của người dùng và hiển thị thông báo thành công.
Luồng ngoại lệ (Exception Flows)	Ngoại lệ 1: AI trả về cấu trúc XML bị lỗi cú pháp - Tại bước 12, Frontend không thể parse được chuỗi JSON bên trong thẻ do AI sinh lỗi cú pháp. - Frontend tự động bỏ qua phần hiển thị Plan Card và chỉ hiển thị nội dung chat dạng text thông thường của AI, kèm theo ghi chú: "Không thể tạo kế hoạch tự động từ phản hồi này, vui lòng yêu cầu chi tiết hơn". Ngoại lệ 2: Dịch vụ NVIDIA NIM API gặp sự cố hoặc timeout - Tại bước 9, Backend gửi request tới NVIDIA NIM nhưng API trả về lỗi HTTP 500 hoặc không phản hồi sau 15 giây. - Backend bắt lỗi bằng block try-catch, trả về mã lỗi HTTP 500 Internal Server Error kèm thông báo: "Dịch vụ AI hiện không phản hồi, vui lòng thử lại sau". Giao diện chat hiển thị tin nhắn báo

	lỗi màu đỏ của hệ thống.
--	--------------------------

3.3.2.4. Use Case: Đồng bộ email tự động nền

- **Mục đích:** Tự động hóa hoàn toàn quy trình chuyển đổi thông tin từ email đối tác/giảng viên thành công việc cần xử lý trong FocusFlow mà không cần bất kỳ thao tác thủ công nào từ phía người dùng.
- **Tình huống thực tế có thể xảy ra:**
 - Hộp thư nhận được nhiều thư quảng cáo, spam không chứa công việc cần làm.
 - Email gửi đến chứa công việc nhưng không ghi rõ hạn hoàn thành (Due Date).
 - Một email đã được đồng bộ ở chu kỳ quét trước, hệ thống quét lại ở chu kỳ này và cần tránh tạo trùng lặp.

***Bảng 3. 6:** Bảng nội dung trong mục Use Case: Đồng bộ email tự động nền*

Thuộc tính	Chi tiết đặc tả
Tên Use Case	Đồng bộ email tự động nền
Actor	Hệ thống tự động (Automation Service), Email Server (IMAP), AI Service (NVIDIA NIM)
Mô tả ngắn	Hệ thống định kỳ kết nối hộp thư Email của người dùng, tự động phân tích và chuyển hóa các email công việc thành các task tương ứng.
Tiền điều kiện	Người dùng cấu hình đầy đủ tài khoản IMAP trong file môi trường .env, hệ thống tự động quét được kích hoạt (ENABLE_AUTOMATION = true).
Hậu điều kiện	Các công việc mới được thêm vào database với nguồn gốc được ghi nhận là Email, kèm theo thông báo in-app gửi đến tài khoản người dùng tương ứng.
Luồng chính (Basic Flow)	1. Định kỳ 5 phút một lần, Timer kích hoạt hàm syncEmails() trong automationService.js. 2. Hệ thống kiểm tra cờ khóa isSyncingEmail để đảm bảo không chạy đè luồng nếu lần quét trước chưa kết thúc. 3. Hệ thống thiết lập kết nối IMAP an toàn (giao thức TLS) tới hộp thư cấu hình. 4. Hệ thống mở thư mục hộp thư đến ("INBOX") và gửi câu lệnh truy vấn tìm kiếm các thư ở trạng thái chưa đọc ("UNSEEN"). 5. IMAP Server trả về danh sách các email chưa đọc kèm theo mã định danh duy nhất (email_uid). 6. Hệ thống thực hiện vòng lặp duyệt qua từng email trong danh sách: a. Gửi truy vấn kiểm tra trong bảng tasks xem

	đã tồn tại bản ghi nào có email_uid này chưa. b. Nếu chưa tồn tại, hệ thống thực hiện tải nội dung chi tiết email và dùng thư viện mailparser để phân tách tiêu đề, người gửi, ngày gửi và nội dung văn bản (body text). c. Hệ thống truyền nội dung văn bản vào hàm isWorkEmail() trong [emailFilter.js](file:///Users/mongnx/Documents/FrontEnd/FocusFlow/backend/src/utils/emailFilter.js). Hàm này sử dụng các mẫu biểu thức chính quy (Regex) và danh sách từ khóa công việc để phân loại. d. Nếu xác định là email công việc, hệ thống gửi nội dung email tới NVIDIA NIM API kèm System Prompt yêu cầu phân tích và trích xuất dữ liệu công việc. e. AI phản hồi về đối tượng JSON chứa các trường: title (tiêu đề task), description (nội dung chi tiết), due_date (thời hạn), và priority (mức ưu tiên). f. Hệ thống tìm kiếm ID người dùng tương ứng dựa trên cấu hình email. Sau đó, thực hiện chèn dữ liệu vào bảng tasks với trường source = 'Email' và email_uid tương ứng. g. Hệ thống tạo thêm một bản ghi thông báo mới trong bảng notifications để báo cho người dùng biết có công việc mới được đồng bộ từ email. 7. Sau khi quét xong toàn bộ danh sách, hệ thống gửi lệnh đánh dấu các email đã xử lý là đã đọc ("SEEN") trên IMAP Server và đóng kết nối.
Luồng ngoại lệ (Exception Flows)	Ngoại lệ 1: Email trùng lặp do quét lại - Tại bước 6a, hệ thống phát hiện email_uid đã tồn tại trong bảng tasks. Hệ thống lập tức bỏ qua email này và chuyển sang email tiếp theo mà không gây lỗi. Ngoại lệ 2: Email là quảng cáo hoặc thư rác (Spam) - Tại bước 6c, hàm lọc isWorkEmail() trả về kết quả false. Hệ thống lập tức bỏ qua email này, tiết kiệm chi phí gọi API AI. Ngoại lệ 3: AI không trích xuất được ngày hạn hoàn thành (Due Date) - Tại bước 6e, nếu AI trả về due_date: null hoặc định dạng ngày không hợp lệ do email không đề cập thời hạn. - Backend tự động tính toán thời hạn mặc định là cuối ngày hôm nay (23:59:59) hoặc 24 giờ kể từ

	thời điểm nhận email để đảm bảo công việc vẫn có thời gian theo dõi. Ngoại lệ 4: Mất kết nối tới IMAP Server - Tại bước 3, việc kết nối IMAP thất bại do sai tài khoản/mật khẩu hoặc lỗi mạng. - Hệ thống ghi nhận lỗi vào log file (console.error) và thoát hàm an toàn. Cờ khóa isSyncingEmail được đặt lại về false để đảm bảo chu kỳ quét tiếp theo (5 phút sau) vẫn được kích hoạt bình thường.
--	---

3.3.2.5. Use Case: AI đề xuất lịch trình & Duyệt kế hoạch

- **Mục đích:** Hỗ trợ người dùng chuyển hóa nhanh chóng các đề xuất lịch trình dạng văn bản của AI thành các công việc thực tế trong hệ thống chỉ bằng một cú click chuột, nâng cao trải nghiệm tự động hóa.
- **Tình huống thực tế có thể xảy ra:**
 - Người dùng đồng ý một phần kế hoạch, bỏ tích chọn một số công việc không cần thiết.
 - Người dùng nhấn chấp nhận nhưng kết nối cơ sở dữ liệu bị ngắt quãng giữa chừng.

Bảng 3. 7: Bảng nội dung trong mục Use Case: AI đề xuất lịch trình & Duyệt kế hoạch

Thuộc tính	Chi tiết đặc tả
Tên Use Case	AI đề xuất lịch trình & Duyệt kế hoạch
Actor	Người dùng (đã xác thực), AI Service (NVIDIA NIM)
Mô tả ngắn	Người dùng duyệt qua các công việc do AI gợi ý trong Plan Card, chỉnh sửa và xác nhận lưu hàng loạt vào cơ sở dữ liệu.
Tiền điều kiện	Trợ lý AI đã trả về phản hồi chứa thẻ định dạng thành công ở giao diện chat.
Hậu điều kiện	Các công việc được chọn lựa sẽ được lưu chính thức vào bảng tasks, hiển thị trên trang Dashboard và Tasks.
Luồng chính (Basic Flow)	1. Trên giao diện chat, Plan Card hiển thị danh sách các task đề xuất do AI phân tích từ cuộc trò chuyện. 2. Mỗi task trong danh sách hiển thị kèm theo Tiêu đề, Thời gian thực hiện, Mức độ ưu tiên và một checkbox ở đầu dòng (mặc định tất cả đều được tích chọn). 3. Người dùng xem xét danh sách, bỏ tích các task không muốn thực hiện. 4. Người dùng nhấn nút "Chấp nhận kế hoạch".

	5. Frontend tổng hợp danh sách các task vẫn còn được tích chọn, tạo ra mảng đối tượng JSON gửi request POST /api/ai/plan/commit lên Backend. 6. Backend tiếp nhận danh sách task kèm theo token xác thực. 7. Backend khởi tạo một Transaction để đảm bảo tính toàn vẹn dữ liệu. 8. Backend duyệt qua mảng task, thực hiện validate thông tin từng task, sau đó thực hiện ghi chép hàng loạt vào bảng tasks liên kết với user_id hiện tại. 9. Database thực hiện lưu trữ thành công toàn bộ bản ghi. 10. Backend commit Transaction và phản hồi HTTP 200 OK kèm số lượng task đã được tạo thành công. 11. Frontend nhận phản hồi, hiển thị thông báo Toast thành công báo số lượng công việc đã thêm, đồng thời reload dữ liệu lịch trình hôm nay.
Luồng ngoại lệ (Exception Flows)	Ngoại lệ 1: Người dùng bỏ tích chọn tất cả các task - Tại bước 3, người dùng bỏ chọn toàn bộ checkbox trên danh sách gợi ý. - Nút "Chấp nhận kế hoạch" sẽ bị vô hiệu hóa (disabled), ngăn chặn việc gửi mảng rỗng lên Server. Ngoại lệ 2: Lỗi ghi dữ liệu hàng loạt trên Database - Tại bước 8, trong quá trình ghi dữ liệu xảy ra lỗi (ví dụ: mất kết nối DB đột ngột). - Backend lập tức thực hiện Rollback Transaction để đưa database về trạng thái an toàn trước đó, không lưu lại dữ liệu dở dang. - Backend phản hồi lỗi HTTP 500 Internal Server Error kèm thông báo lỗi. Frontend hiển thị thông báo: "Không thể lưu kế hoạch do lỗi hệ thống, vui lòng thử lại".

3.3.2.6. Use Case: Nhắc nhở chủ động qua Discord

- **Mục đích:** Tận dụng kênh giao tiếp bên ngoài (Discord) để gửi cảnh báo trực tiếp, giúp người dùng nắm bắt công việc sắp đến hạn mà không bị phụ thuộc vào việc phải liên tục mở tab trình duyệt ứng dụng FocusFlow.
- **Tình huống thực tế có thể xảy ra:**
 - Người dùng chưa cấu hình Discord Webhook URL trong hệ thống.
 - Webhook URL bị cấu hình sai hoặc bị người dùng xóa phía bên Discord.

Website Lập Lịch và Quản Lý Công Việc

- Công việc sắp đến hạn nhưng người dùng vừa thực hiện tích hoàn thành (Done) ngay trước thời điểm quét.

Bảng 3. 8: Bảng nội dung trong mục Use Case: Nhắc nhở chủ động qua Discord

Thuộc tính	Chi tiết đặc tả
Tên Use Case	Nhắc nhở chủ động qua Discord
Actor	Hệ thống tự động (Automation Service), Discord Webhook API
Mô tả ngắn	Hệ thống quét dữ liệu công việc sắp đến hạn và tự động gửi tin nhắn nhắc nhở trực tiếp vào kênh Discord của người dùng.
Tiền điều kiện	Người dùng đã thiết lập cấu hình biến môi trường DISCORD_WEBHOOK_URL trong hệ thống.
Hậu điều kiện	Người dùng nhận được tin nhắn thông báo trên Discord, cờ discord_notified của task trong database chuyển sang 1.
Luồng chính (Basic Flow)	1. Định kỳ mỗi 1 phút, Timer gọi hàm checkAndSendDiscordReminders() trong automationService.js. 2. Hệ thống kiểm tra cờ khóa isCheckingDiscord để tránh chạy lặp chồng chéo. 3. Hệ thống tính toán khoảng thời gian quét (Cửa sổ thời gian): từ 10 phút trước đến 30 phút sau thời điểm hiện tại (now). Khoảng thời gian 10 phút trước giúp đảm bảo không bỏ sót task nếu hệ thống vừa được khởi động lại. 4. Hệ thống thực hiện truy vấn database lấy các công việc thỏa mãn 4 điều kiện cùng lúc: - Trạng thái công việc khác 'done' (status != 'done'). - Cờ nhắc nhở bằng 0 (discord_notified = 0). - Trường ngày hạn không rỗng (due_date IS NOT NULL). - Ngày hạn nằm trong cửa sổ thời gian đã tính toán ở bước 3. 5. Database trả về danh sách các task thỏa mãn điều kiện. 6. Với mỗi task trong danh sách, hệ thống soạn thảo nội dung tin nhắn dạng Markdown bao gồm: - Biểu tượng cảnh báo khẩn cấp tương ứng với mức độ ưu tiên. - Tiêu đề công việc, Mô tả ngắn. - Thời hạn hoàn thành chi tiết và Nguồn gốc công việc. 7. Hệ thống gửi yêu cầu HTTP POST đến DISCORD_WEBHOOK_URL chứa nội dung

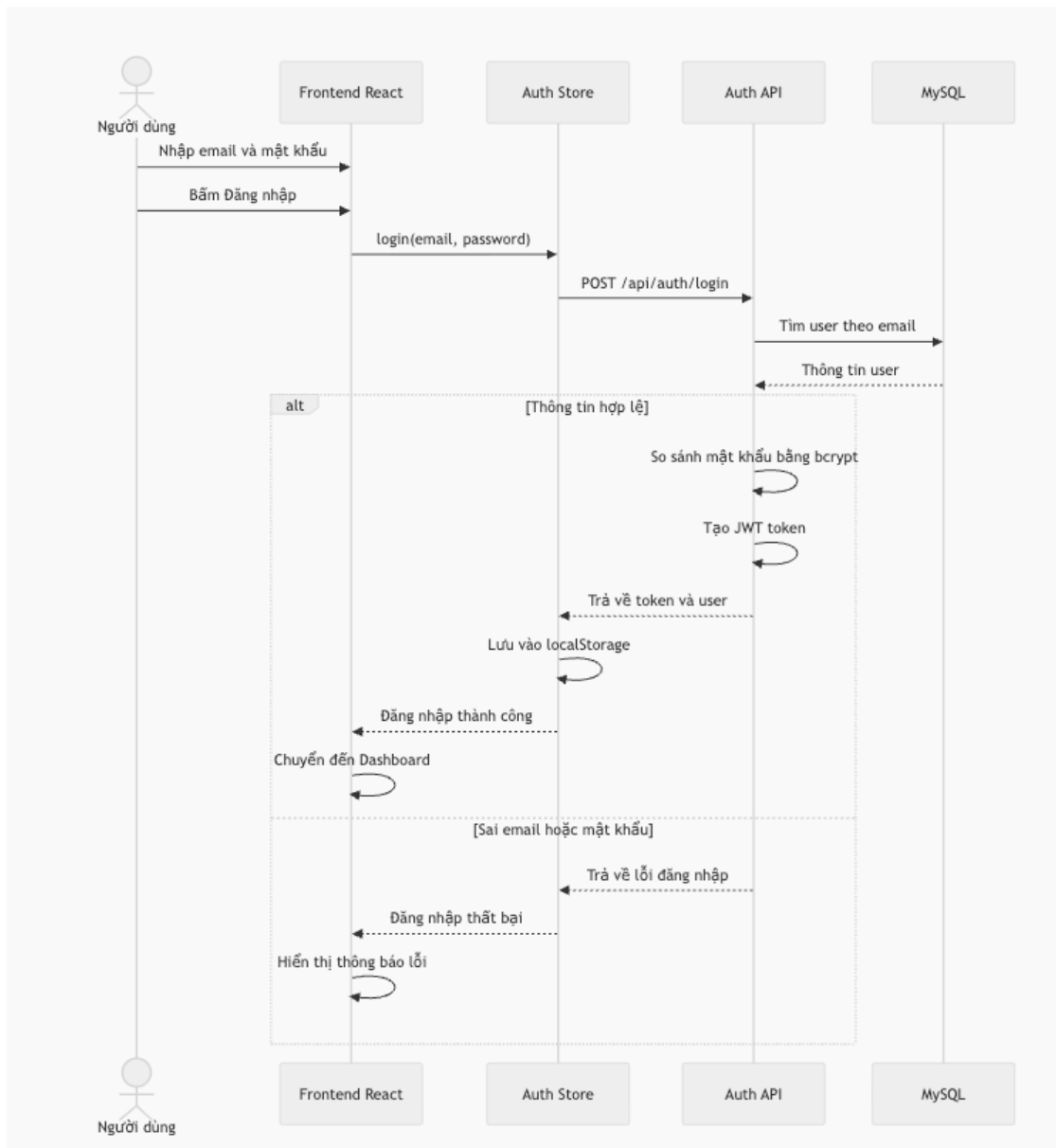
	tin nhắn dạng JSON payload. 8. Discord API tiếp nhận request, xác thực URL hợp lệ, hiển thị tin nhắn trong kênh chat và phản hồi HTTP 200 OK hoặc 204 No Content về Server. 9. Backend nhận phản hồi thành công, lập tức thực hiện câu lệnh UPDATE tasks SET discord_notified = 1 WHERE id = ? để cập nhật trạng thái đã thông báo cho công việc này.
Luồng ngoại lệ (Exception Flows)	Ngoại lệ 1: Thiếu cấu hình Webhook URL - Tại bước 2, hệ thống phát hiện biến DISCORD_WEBHOOK_URL trong file cấu hình không tồn tại hoặc để trống. - Hệ thống lập tức thoát hàm an toàn và ghi log cảnh báo nhẹ, không thực hiện quét DB vô ích. Ngoại lệ 2: Gửi request thất bại do Webhook URL bị hỏng hoặc hết hạn - Tại bước 7, Discord trả về lỗi HTTP 404 Not Found hoặc 401 Unauthorized do webhook bị xóa bên phía Discord. - Hệ thống bắt lỗi trong block try-catch, ghi log chi tiết lỗi, giữ nguyên cờ discord_notified = 0 của task để hệ thống có thể thử lại sau khi người dùng sửa lại cấu hình. Ngoại lệ 3: Công việc đã được hoàn thành trước thời điểm gửi - Nếu người dùng đánh dấu hoàn thành công việc ở giao diện web ngay lúc hệ thống đang chuẩn bị gửi tin nhắn. - Do câu lệnh truy vấn ở bước 4 có điều kiện status != 'done' nên công việc này sẽ tự động bị loại bỏ khỏi danh sách nhắc nhở ở bước 5, tránh gây phiền nhiễu cho người dùng.

3.4. Biểu đồ tuần tự (Sequence Diagram)

Phần này mô tả dòng tương tác động giữa các thành phần phần mềm của FocusFlow qua 5 biểu đồ tuần tự tương ứng với các quy trình nghiệp vụ cốt lõi. Mỗi biểu đồ đi kèm với sự phân tích logic chi tiết bước đi của dữ liệu.

3.4.1. Quy trình Đăng nhập hệ thống

Biểu đồ tuần tự dưới đây mô tả quá trình từ khi người dùng nhập thông tin đăng nhập trên giao diện web cho đến khi được cấp quyền và chuyển hướng vào Dashboard:



Hình 3. 3: Sơ đồ minh họa cho Quy trình Đăng nhập hệ thống

Giải thích chi tiết các bước tương tác:

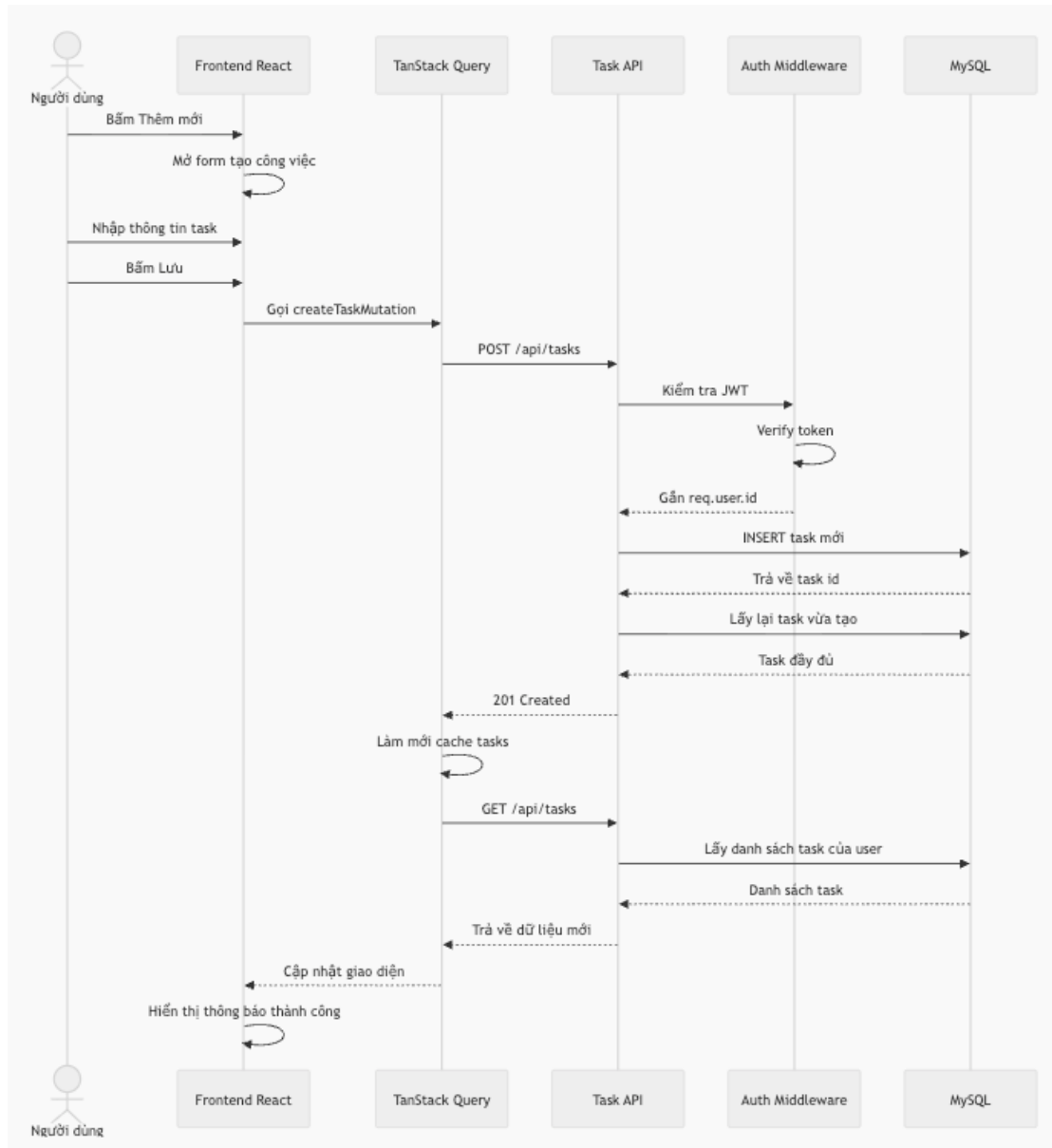
1. **Khởi đầu tương tác:** Người dùng điền thông tin và nhấn "Đăng nhập" trên Form. Frontend React tiếp nhận sự kiện, thực hiện gọi hàm `login` nằm trong `AuthStore` của Zustand.

Website Lập Lịch và Quản Lý Công Việc

2. **Gửi yêu cầu mạng:** AuthStore thực hiện một request HTTP POST bất đồng bộ (`axios.post`) đến endpoint `/api/auth/login` mang theo payload chứa email và mật khẩu thô.
3. **Truy vấn cơ sở dữ liệu:** Backend Express đón nhận request tại router, chuyển tiếp đến `authController`. Controller thực hiện gọi connection pool của MySQL chạy câu lệnh truy vấn có tham số (Prepared Statement) để tìm người dùng theo Email.
4. **Xác thực mật khẩu & Tạo Token:**
 - Sau khi nhận được bản ghi người dùng từ DB, Controller dùng thư viện `bcrypt` so sánh mật khẩu người dùng vừa gửi với mật khẩu đã mã hóa lưu trong DB.
 - Nếu so sánh khớp, Controller tiến hành sinh JWT Token sử dụng hàm `jwt.sign`. Token được ký bằng khóa bí mật lấy từ biến môi trường `process.env.JWT_SECRET`, thời hạn sống của token được thiết lập là 7 ngày.
5. **Cập nhật trạng thái Client:**
 - Token và thông tin người dùng được Backend trả về cho Client.
 - Zustand Store lưu trữ JWT Token và thông tin người dùng vào `localStorage` của trình duyệt nhằm duy trì phiên đăng nhập khi người dùng F5 tải lại trang. Đồng thời cập nhật trạng thái xác thực trong RAM `isAuthenticated = true`.
6. **Điều hướng giao diện:** Frontend nhận trạng thái thành công, gọi thư viện `react-hot-toast` hiển thị thông báo chào mừng mừng mà, sau đó dùng `useNavigate` chuyển hướng người dùng đến `/dashboard`.
7. **Xử lý lỗi (Nhánh Else):** Nếu truy vấn không ra user hoặc mật khẩu không khớp, backend phản hồi status code `400 Bad Request` kèm thông báo lỗi chung. Store nhận lỗi và báo về component. Component hiển thị Toast màu đỏ để cảnh báo người dùng.

3.4.2. Quy trình Tạo công việc mới

Biểu đồ tuần tự dưới đây thể hiện quy trình tạo công việc thủ công, làm nổi bật vai trò của Middleware xác thực JWT và cơ chế tự động đồng bộ lại cache của



Hình 3. 4: Sơ đồ minh họa cho Quy trình Tạo công việc mới

Giải thích chi tiết các bước tương tác:

- Thiết lập giao diện:** Người dùng nhấn "Thêm mới" mở modal Form. Sau khi điền đủ thông tin và nhấn "Lưu", Frontend gọi phương thức `mutate` của đối tượng Mutation được định nghĩa bởi thư viện TanStack Query.
- Xác thực Token tại Middleware:** Request HTTP POST được gửi tới `/api/tasks`. Điểm đặc biệt ở đây là request đi qua `authMiddleware` trước khi vào controller. Middleware lấy JWT từ header `Authorization: Bearer`, gọi hàm `jwt.verify` để giải mã. Nếu token hợp lệ, middleware trích xuất ID người dùng và đính kèm

Website Lập Lịch và Quản Lý Công Việc

trực tiếp vào đối tượng `req.user`, sau đó gọi hàm `next()` để chuyển tiếp quyền xử lý sang `taskController`.

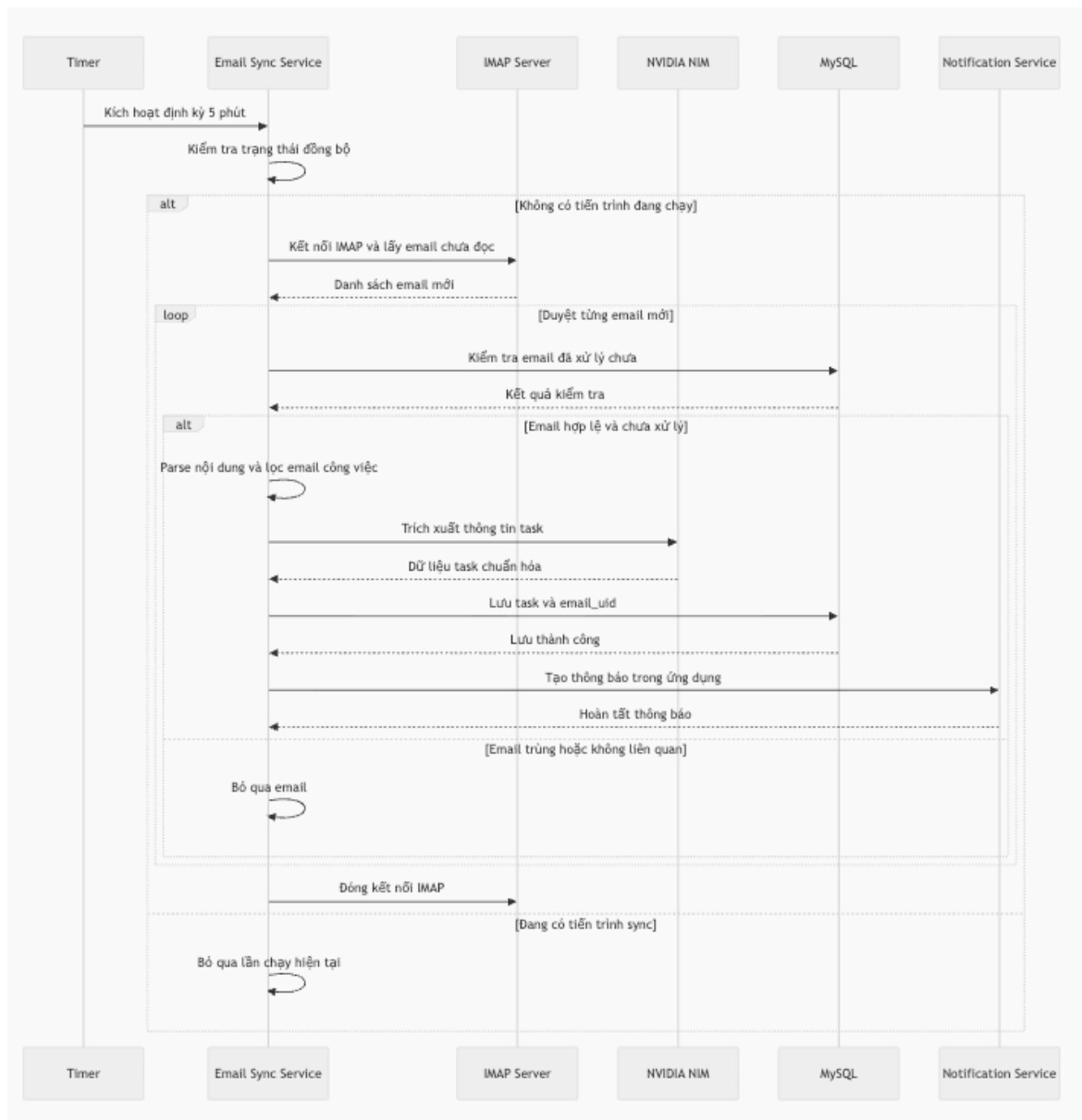
3. **Lưu trữ dữ liệu:** `taskController` nhận dữ liệu đã được giải mã thông tin `user`, thực hiện câu lệnh SQL `INSERT INTO tasks` để thêm mới bản ghi. Sau khi thêm thành công, hệ thống thực hiện thêm một câu truy vấn `SELECT` phụ để lấy lại toàn bộ cấu trúc đầy đủ của task vừa tạo (bao gồm cả các giá trị default mà DB tự sinh) trả về cho Client với mã trạng thái 201 `Created`.

4. Đồng bộ hóa trạng thái phía Client:

- Thay vì phải viết code thủ công để chèn task mới vào mảng dữ liệu hiện tại ở giao diện, ứng dụng sử dụng tính năng **Invalidate Cache** của TanStack Query: `invalidateQueries(['tasks'])`.
- Lệnh này đánh dấu bộ dữ liệu công việc hiện tại là lỗi thời (stale), kích hoạt một luồng fetch dữ liệu ngầm (background refetch) gọi API `GET /api/tasks` để tải lại danh sách mới nhất từ Database.
- Giao diện người dùng lập tức tự động cập nhật hiển thị task mới một cách mượt mà và hiển thị thông báo Toast thành công.

3.4.3. Quy trình Đồng bộ email tự động nền

Biểu đồ này biểu diễn quy trình chạy nền tuần kỳ, minh họa sự phối hợp giữa Background Worker, Email IMAP Server, NVIDIA NIM AI và Database:



Hình 3. 5: Sơ đồ minh họa cho Quy trình Đồng bộ email tự động nền (UC19)

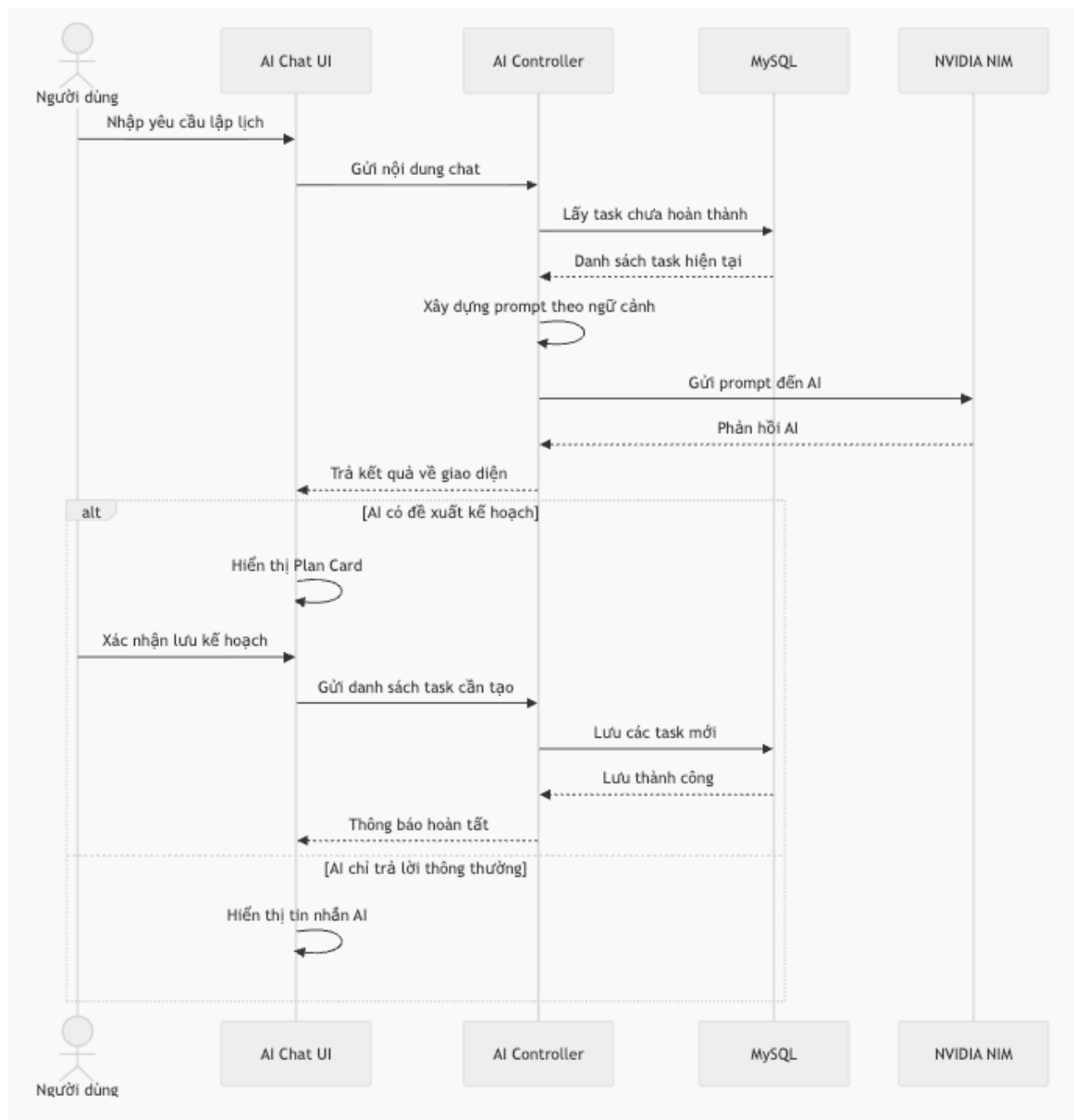
Giải thích chi tiết các bước tương tác và Quyết định Thiết kế:

1. **Tránh tranh chấp tài nguyên (Cờ khóa):** Do dịch vụ chạy định kỳ mỗi 5 phút bằng `setInterval`, trong trường hợp hòm thư có quá nhiều email hoặc API AI phản hồi chậm vượt quá 5 phút, chu kỳ tiếp theo có thể bị kích hoạt đè lên chu kỳ cũ, gây xung đột kết nối IMAP và ghi trùng dữ liệu. Để giải quyết, `automationService.js` sử dụng một biến cờ trạng thái `isSyncingEmail`. Khi bắt đầu chạy, cờ được đặt thành `true` và chặn mọi yêu cầu quét mới cho đến khi quy trình hiện tại kết thúc hoàn toàn và cờ chuyển về `false`.

2. **Kết nối và Tìm kiếm:** Dịch vụ kết nối IMAP qua thư viện `node-imap`. Sau khi xác thực thành công, hệ thống tìm kiếm các mail có cờ `UNSEEN` (Chưa đọc) và lấy về mảng đối tượng.
3. **Lọc thông tin trùng lặp ở tầng Database:** Để đảm bảo tính Idempotency (chạy nhiều lần không đổi kết quả), hệ thống sử dụng ID duy nhất của email trên server thư làm khóa kiểm tra (`email_uid`). Trước khi xử lý tiếp, hệ thống gửi lệnh `SELECT` để xác thực email này chưa từng được lưu trong cơ sở dữ liệu.
4. **Tiết kiệm chi phí gọi AI bằng bộ lọc Regex địa phương:** Thay vì gửi trực tiếp toàn bộ email chưa đọc sang API AI (tốn tài nguyên mạng và chi phí API), hệ thống đưa nội dung qua bộ lọc. Bộ lọc này phân tích tiêu đề và nội dung để loại bỏ các email spam, quảng cáo phổ biến. Chỉ những email thực sự chứa nội dung công việc mới được chuyển tiếp sang bước gọi AI.
5. **Gọi AI trích xuất thực thể:** Backend gửi nội dung email công việc thô đến NVIDIA NIM API kèm theo hướng dẫn cụ thể để mô hình trả về dữ liệu đúng định dạng cấu trúc JSON định sẵn.
6. **Lưu trữ và Tạo thông báo:** Hệ thống thực hiện truy vấn chuyển đổi email người nhận thành ID người dùng tương ứng trong DB, lưu trữ công việc mới vào bảng `tasks`, đồng thời tạo một thông báo mới trong bảng `notifications` để hiển thị trên giao diện in-app của người dùng. Kết thúc quy trình, đóng kết nối IMAP an toàn.

3.4.4. Quy trình Chat và Lập kế hoạch với AI

Biểu đồ này thể hiện sự tương tác thời gian thực khi người dùng trò chuyện bằng ngôn ngữ tự nhiên để AI phân tích và đề xuất một kế hoạch gồm nhiều công việc dưới dạng giao diện trực quan:



Hình 3. 6: Sơ đồ minh họa cho Quy trình Chat và Lập kế hoạch với AI

Giải thích chi tiết các bước tương tác và Xử lý lỗi:

- Thiết lập Prompt động thông minh (Context Injection):** Khi nhận yêu cầu chat từ người dùng, aiController không gửi trực tiếp câu chat lên AI. Để AI đưa ra câu trả lời thực sự hữu ích, hệ thống truy vấn MySQL lấy toàn bộ danh sách công việc hiện có của người dùng và chèn vào System Prompt. Điều này giúp AI có trí nhớ ngắn hạn về khối lượng công việc hiện tại của người dùng.
- Truy vấn Email theo nhu cầu (On-demand Email Fetching):** Hệ thống thông minh phát hiện nếu trong tin nhắn của người dùng chứa các từ khóa như "thư", "email", hệ thống sẽ chủ động thực hiện kết nối IMAP thời gian thực để kéo về nội

Website Lập Lịch và Quản Lý Công Việc

dung của 5 email công việc gần nhất để chuyển cho AI làm tư liệu trả lời câu hỏi của người dùng.

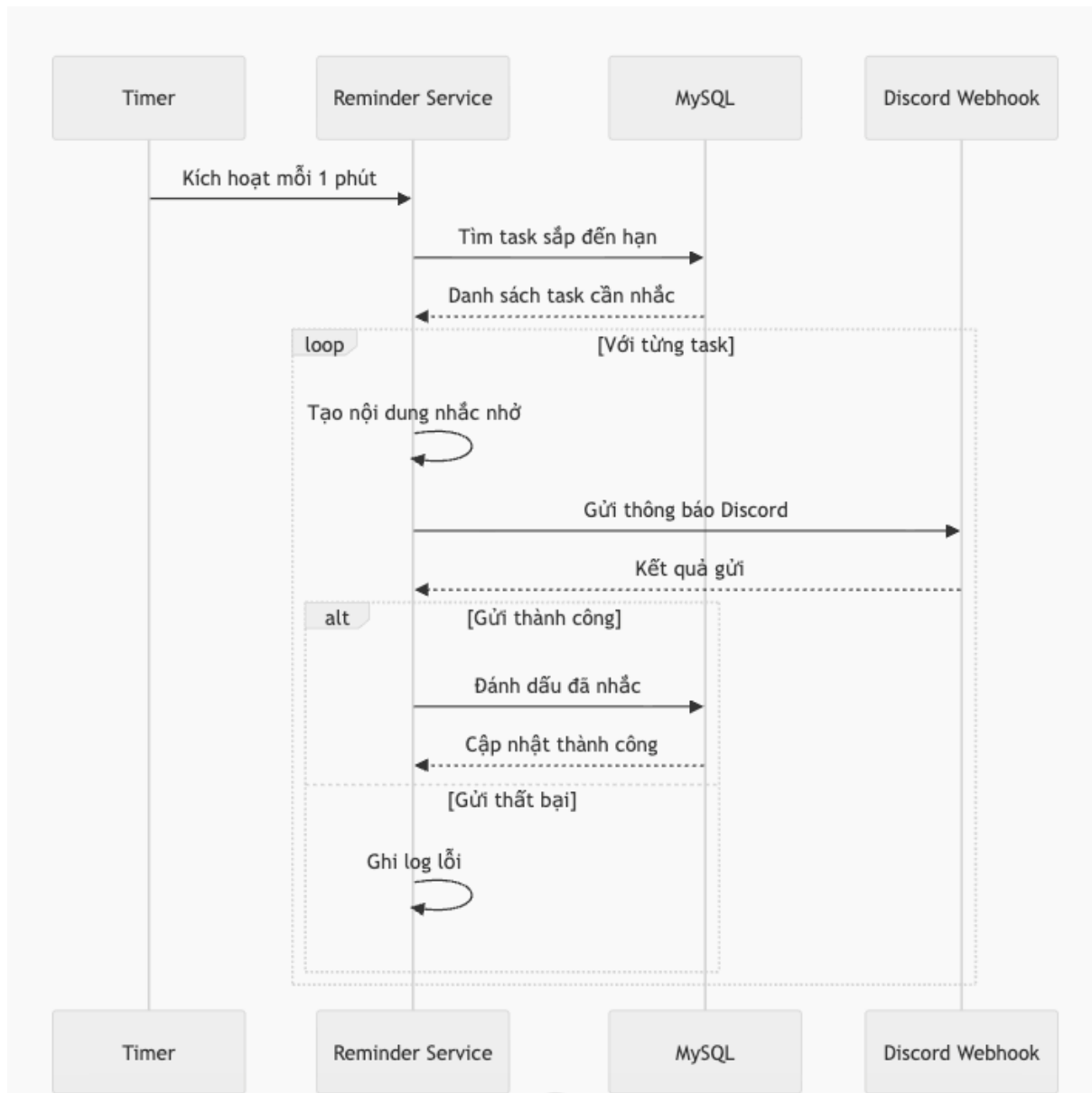
3. Phân tích cú pháp phía Client (Strict XML Parsing):

- AI được cấu hình xuất ra định dạng thẻ XML ... bao quanh chuỗi JSON chứa danh sách task đề xuất.
- Tại Frontend, hệ thống sử dụng biểu thức chính quy (Regex) `/([\s\S]*?)/` để tìm kiếm sự hiện diện của thẻ này.
- Nếu tìm thấy, Frontend tách phần text thường để hiển thị vào bong bóng chat, đồng thời parse chuỗi JSON nằm trong thẻ plan thành một mảng đối tượng JavaScript và truyền vào component `PlanCard` để hiển thị danh sách các task đề xuất một cách trực quan, sinh động.

4. **Duyệt kế hoạch hàng loạt (Batch Commit):** Khi người dùng đồng ý và bấm nút chấp nhận kế hoạch, mảng các task được gửi lên endpoint `/api/ai/plan/commit`. Backend dùng vòng lặp để ghi chép toàn bộ vào DB trong một Transaction để đảm bảo tính nhất quán (nếu một task ghi lỗi, toàn bộ transaction bị hủy để tránh dữ liệu rác).

3.4.5. Quy trình Nhắc nhở qua Discord

Biểu đồ mô tả hoạt động chạy nền định kỳ mỗi phút của hệ thống để phát hiện các công việc sắp đến hạn và chủ động đẩy cảnh báo ra bên ngoài:



Hình 3. 7: Sơ đồ minh họa cho Quy trình Nhắc nhở qua Discord

Giải thích chi tiết các bước tương tác:

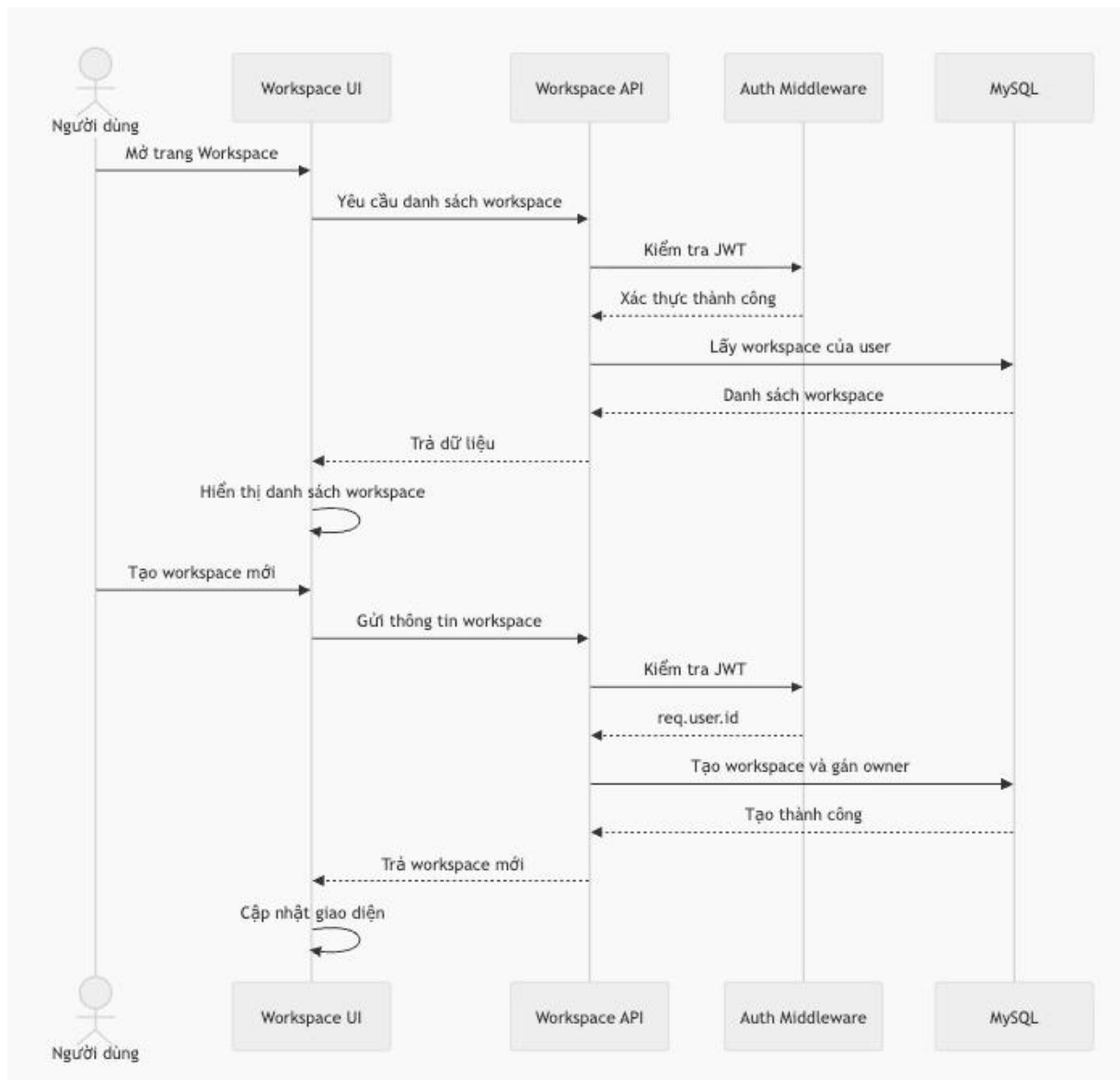
- Chu kỳ kích hoạt:** Tần suất quét được thiết lập là 1 phút một lần thông qua Timer của NodeJS. Cờ `isCheckingDiscord` được bật để đảm bảo chỉ có duy nhất một tiến trình nhắc nhở chạy tại một thời điểm.
- Cửa sổ thời gian an toàn (Time Windowing):**

Website Lập Lịch và Quản Lý Công Việc

- Thay vì chỉ quét các task có deadline đúng bằng thời điểm hiện tại, hệ thống tính toán một cửa sổ thời gian linh hoạt bắt đầu từ `now - 10 phút` đến `now + 30 phút`.
 - Việc bắt đầu từ 10 phút trước đảm bảo tính bền bỉ của hệ thống (resiliency). Nếu server bị tắt để bảo trì hoặc mất điện trong 5 phút và khởi động lại, các task có hạn rơi vào đúng khoảng thời gian server sập vẫn sẽ được quét và gửi nhắc nhở bù, không bị trôi mất.
3. **Lọc dữ liệu chưa gửi:** Dữ liệu truy vấn từ DB chỉ chọn những task chưa hoàn thành (`status != 'done'`) và chưa từng gửi thông báo thành công (`discord_notified = 0`).
4. **Gửi tin nhắn định dạng:**
- Hệ thống soạn thảo chuỗi Markdown tương thích với Discord API, tạo ra giao diện thông báo chuyên nghiệp.
 - Gửi request POST dạng JSON payload đến Webhook URL.
5. **Chốt trạng thái gửi (Cờ an toàn):** Chỉ khi nhận được phản hồi HTTP thành công (`status 200` hoặc `204`) từ Discord API, hệ thống mới cập nhật cờ `discord_notified = 1` trong MySQL cho task tương ứng. Điều này ngăn chặn việc đánh dấu ảo (đã đánh dấu là gửi thành công nhưng thực tế chưa gửi được do nghẽn mạng). Nếu gửi lỗi, cờ giữ nguyên là 0 để hệ thống tự động quét lại và gửi lại ở phút tiếp theo.

3.4.6. Quy trình Quản lý Workspace và Giao việc nhóm

Biểu đồ mô tả hoạt động mời thành viên mới vào không gian làm việc và Owner/Admin giao việc (assign) cho các thành viên trong nhóm:



Hình 3. 8: Sơ đồ minh họa cho Quy trình Quản lý Workspace và Giao việc nhóm

Giải thích chi tiết các bước tương tác:

- Quản lý phiên lời mời an toàn:** Khi Owner gửi lời mời, hệ thống không cấp quyền ngay lập tức mà tạo ra một token bảo mật ngẫu nhiên mã hóa có thời hạn lưu trong bảng `workspace_invites`.
- Xác nhận danh tính đa bước:** Thành viên nhận được token qua Email, khi nhấn vào liên kết, hệ thống yêu cầu thành viên phải đăng nhập (để xác nhận `user_id`). Sau đó, backend đối chiếu token và chuyển trạng thái lời mời sang `accepted`.
- Phân quyền và Phân công (Assigning Tasks):** Owner tạo công việc, truyền lên `workspace_id` và `assigned_to` là ID của Member mới. Thành viên mới ngay lập

Website Lập Lịch và Quản Lý Công Việc

tức nhìn thấy thẻ công việc được gán tên mình hiển thị trên bảng WorkspaceTaskBoard.

3.5. Thiết kế cơ sở dữ liệu

Hệ thống FocusFlow sử dụng hệ quản trị cơ sở dữ liệu quan hệ MySQL để quản lý tính nhất quán và toàn vẹn của dữ liệu người dùng.

3.5.1. Danh sách các bảng dữ liệu

Cơ sở dữ liệu bao gồm 8 bảng chính được tối ưu hóa chuẩn hóa dữ liệu:

Bảng 3. 9: Bảng nội dung trong mục Danh sách các bảng dữ liệu

STT	Tên Bảng	Vai trò nghiệp vụ	Số lượng thuộc tính
1	users	Lưu trữ thông tin định danh, tài khoản đăng nhập và cấu hình của người dùng.	13
2	tasks	Lưu trữ thông tin chi tiết về các công việc, trạng thái, mức độ ưu tiên và nguồn.	15
3	events	Lưu trữ lịch hẹn, sự kiện dạng khoảng thời gian hiển thị trên Calendar.	9
4	notifications	Lưu trữ nhật ký thông báo tự động để hiển thị in-app.	9
5	workspaces	Lưu trữ thông tin không gian làm việc chung (nhóm).	7
6	workspace_members	Quản lý thành viên và phân quyền (owner, admin, member) trong workspace.	5
7	workspace_invites	Lưu trữ lời mời tham gia workspace qua email bằng token bảo mật.	7
8	task_comments	Lưu trữ bình luận tương tác của các thành viên trên từng công việc chung.	5

3.5.2. Chi tiết cấu trúc từng bảng dữ liệu

- Bảng users (Thông tin người dùng)

Website Lập Lịch và Quản Lý Công Việc

Bảng này chứa dữ liệu tài khoản hệ thống hoặc tài khoản đồng bộ từ liên kết Google OAuth.

Bảng 3. 10: Bảng nội dung trong mục Bảng users (Thông tin người dùng)

Tên trường	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Giải thích ý nghĩa
id	INT	Primary Key, Auto Increment	Không	Mã số định danh duy nhất của người dùng
google_id	VARCHAR(255)	Unique Key, Nullable	NULL	ID định danh duy nhất do Google cấp (dùng khi login Google)
username	VARCHAR(255)	NOT NULL	Không	Tên tài khoản người dùng
email	VARCHAR(255)	Unique Key, NOT NULL	Không	Địa chỉ email liên lạc và dùng để đăng nhập
password	VARCHAR(255)	Nullable	NULL	Chuỗi mật khẩu băm bcrypt (để NULL nếu dùng Google Login)
display_name	VARCHAR(255)	Nullable	NULL	Tên hiển thị trên giao diện người dùng
avatar_url	VARCHAR(500)	Nullable	NULL	Đường dẫn ảnh đại diện của người dùng
created_at	TIMESTAMP	NOT NULL	CURRENT_TIME STAMP	Thời gian tài khoản được khởi tạo

- Bảng tasks (Thông tin công việc)

Bảng trung tâm của hệ thống, chứa thông tin công việc từ mọi nguồn đổ về.

Bảng 3. 11: Bảng nội dung trong mục Bảng tasks (Thông tin công việc)

Tên trường	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Giải thích ý nghĩa
id	INT	Primary Key, Auto Increment	Không	Mã số công việc duy nhất
user_id	INT	Foreign Key -> users(id), NOT NULL	Không	Liên kết xác định công việc thuộc về người dùng nào
title	VARCHAR(255)	NOT NULL	Không	Tiêu đề công việc
description	TEXT	Nullable	NULL	Mô tả chi tiết các bước cần thực hiện
priority	VARCHAR(50)	NOT NULL	'2'	Mức ưu tiên: '1' (Cao), '2' (Trung bình), '3' (Thấp)
status	VARCHAR(50)	NOT NULL	'todo'	Trạng thái: 'todo' (Cần làm),

Website Lập Lịch và Quản Lý Công Việc

				'in_progress', 'done' (Hoàn thành)
due_date	DATETIME	Nullable	NULL	Ngày và giờ hạn chót cần hoàn thành
category_id	INT	Nullable	NULL	Mã liên kết danh mục phân loại công việc
source	VARCHAR(50)	NOT NULL	'Custom'	Nguồn tạo: 'Custom', 'Email', 'AI', 'Slack', 'Discord'
email_uid	VARCHAR(255)	Unique Key, Nullable	NULL	ID email IMAP độc bản (tránh trùng lặp khi đồng bộ nền)
discord_notified	TINYINT(1)	NOT NULL	0	Trạng thái nhắc nhở: 0 (Chưa thông báo), 1 (Đã thông báo)
updated_at	TIMESTAMP	ON UPDATE	CURRENT_TIME STAMP	Thời điểm công việc được cập nhật lần cuối cùng
created_at	TIMESTAMP	NOT NULL	CURRENT_TIME STAMP	Thời điểm công việc được thêm vào hệ thống

- Bảng events (Sự kiện lịch hẹn)

Phục vụ tính năng lịch trình, lưu trữ các sự kiện có mốc thời gian bắt đầu và kết thúc cụ thể.

Bảng 3. 12: Bảng nội dung trong mục Bảng events (Sự kiện lịch hẹn)

Tên trường	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Giải thích ý nghĩa
id	INT	Primary Key, Auto Increment	Không	Mã số sự kiện duy nhất
user_id	INT	Foreign Key -> users(id), NOT NULL	Không	Liên kết xác định sự kiện thuộc người dùng nào
title	VARCHAR(255)	NOT NULL	Không	Tiêu đề của sự kiện hoặc buổi họp
description	TEXT	Nullable	NULL	Mô tả chi tiết nội dung sự kiện
start_time	DATETIME	NOT NULL	Không	Mốc thời gian bắt đầu sự kiện
end_time	DATETIME	NOT NULL	Không	Mốc thời gian kết thúc sự kiện
source	VARCHAR(50)	NOT NULL	'Custom'	Nguồn gốc tạo sự kiện
created_at	TIMESTAMP	NOT NULL	CURRENT_TIME STAMP	Thời điểm sự kiện được khởi tạo

- Bảng notifications (Nhật ký thông báo hệ thống)

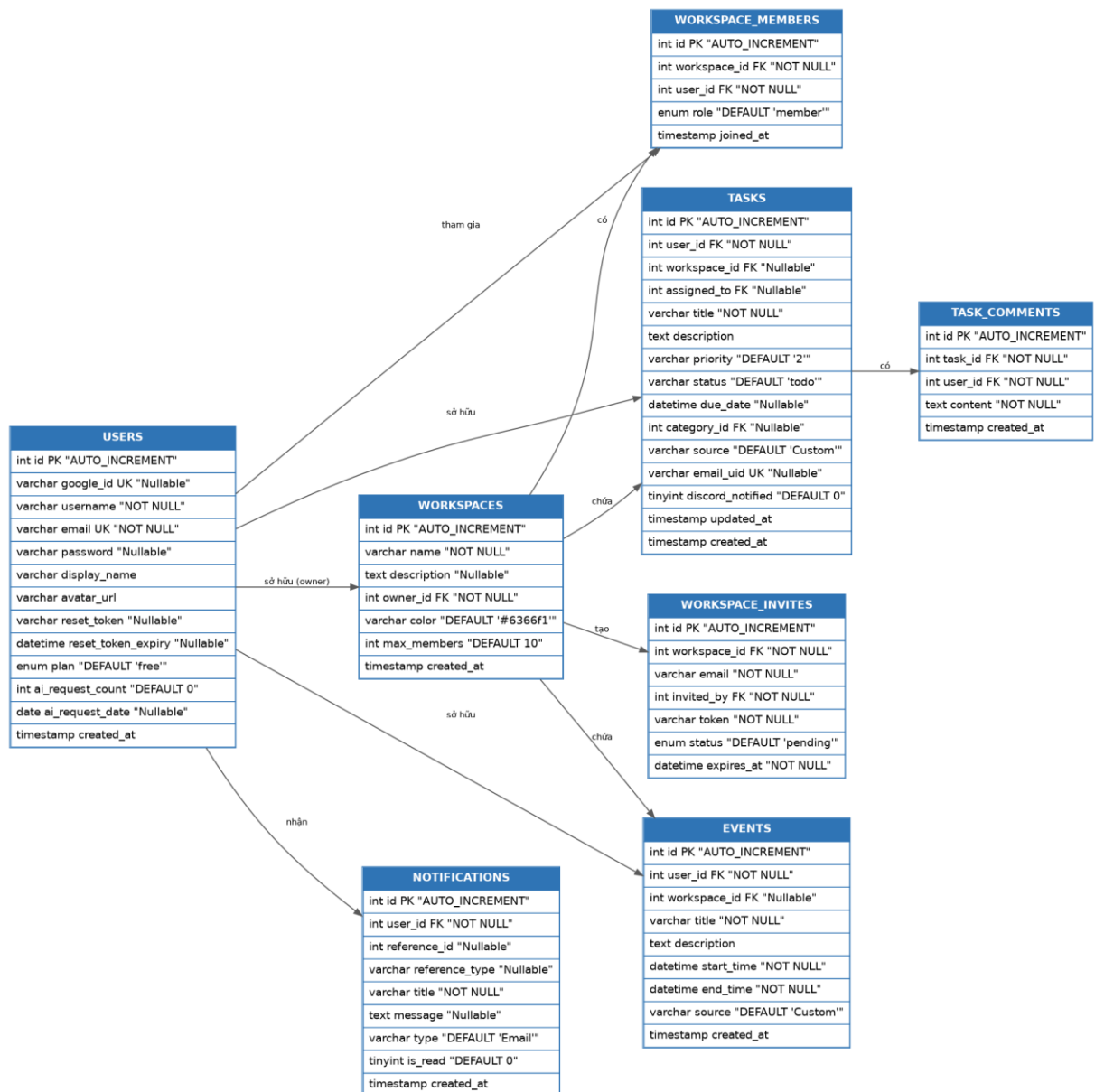
Lưu trữ lịch sử thông báo in-app cho người dùng khi hệ thống tự động hoàn thành các tác vụ nền.

Bảng 3. 13: Bảng nội dung trong mục Bảng notifications (Nhật ký thông báo hệ thống)

Tên trường	Kiểu dữ liệu	Ràng buộc	Giá trị mặc định	Giải thích ý nghĩa
id	INT	Primary Key, Auto Increment	Không	Mã số thông báo duy nhất
user_id	INT	Foreign Key -> users(id), NOT NULL	Không	Xác định người dùng nhận thông báo này
title	VARCHAR(255)	NOT NULL	Không	Tiêu đề thông báo ngắn gọn
message	TEXT	Nullable	NULL	Nội dung chi tiết của thông báo
type	VARCHAR(50)	NOT NULL	'Email'	Phân loại: 'Email' (Đồng bộ thư), 'task', 'system'
is_read	TINYINT(1)	NOT NULL	0	Trạng thái xem: 0 (Chưa đọc), 1 (Đã đọc)
created_at	TIMESTAMP	NOT NULL	CURRENT_TIME STAMP	Thời điểm thông báo được sinh ra

- Biểu đồ quan hệ thực thể (ERD)

Biểu đồ thực thể mô tả mối quan hệ thể hiện tính toàn vẹn dữ liệu tham chiếu giữa các thực thể trong cơ sở dữ liệu của FocusFlow:



Hình 3. 9: Sơ đồ minh họa cho Biểu đồ quan hệ thực thể (ERD)

Mối quan hệ và ràng buộc dữ liệu:

- Quan hệ USERS - TASKS / EVENTS / NOTIFICATIONS (1 - Nhiều):** Liên kết thông qua trường `user_id` tham chiếu đến `users.id`. Ràng buộc này đảm bảo mọi bản ghi đều phải thuộc về một người dùng xác định, với `ON DELETE CASCADE` được thiết lập. Khi người dùng bị xóa, mọi công việc, sự kiện và thông báo cá nhân sẽ được dọn dẹp.
- Quan hệ WORKSPACES - USERS (1 - Nhiều):** Bảng `workspaces` liên kết với `users.id` qua `owner_id`, đánh dấu chủ sở hữu. Bảng trung gian `workspace_members` kết nối nhiều `user_id` với một `workspace_id`, quản lý phân

Website Lập Lịch và Quản Lý Công Việc

quyền (owner/admin/member). Bảng `workspace_invites` kết nối với `users` qua `invited_by`.

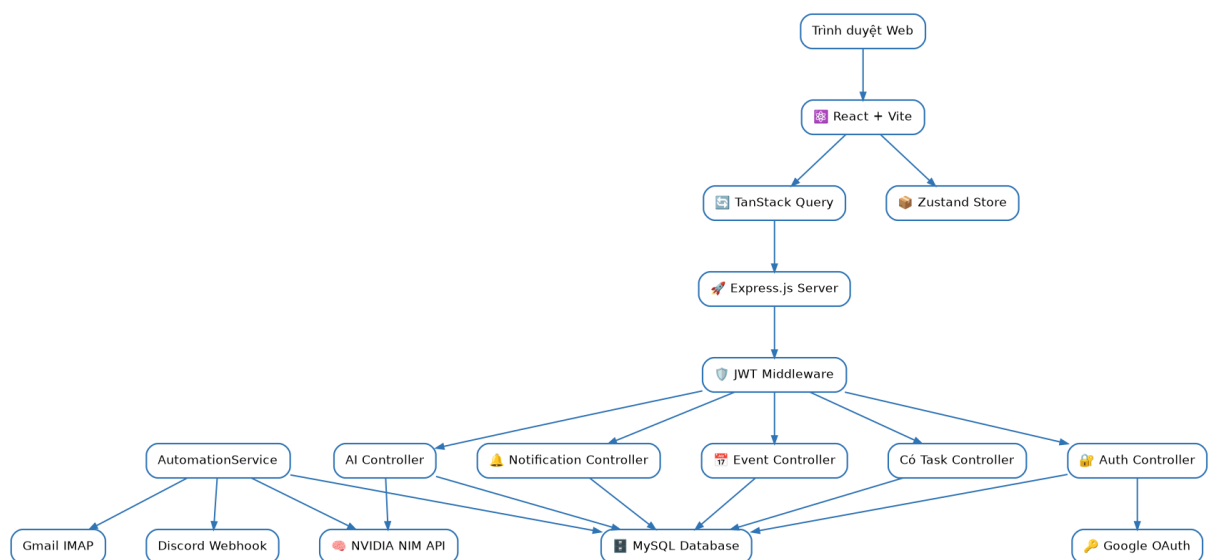
3. **Quan hệ WORKSPACES - TASKS / EVENTS (1 - Nhiều):** Bảng `tasks` và `events` bổ sung trường `workspace_id` và `assigned_to` (dành cho `Tasks`) để phân công. Cấu hình `ON DELETE SET NULL` giúp giữ lại công việc cá nhân khi thành viên rời khỏi `Workspace`, hoặc `CASCADE` để xóa công việc chung khi `Workspace` bị hủy bỏ tùy theo nghiệp vụ.

4. **Quan hệ TASKS - TASK_COMMENTS (1 - Nhiều):** Bình luận của người dùng trên mỗi công việc (phục vụ cộng tác nhóm) được liên kết qua `task_id` và `user_id` với cơ chế xóa `ON DELETE CASCADE`.

3.6. Thiết kế kiến trúc hệ thống

3.6.1. Kiến trúc 3 tầng (3-tier Architecture)

Hệ thống FocusFlow được thiết kế tuân thủ nghiêm ngặt mô hình kiến trúc 3 tầng chuẩn hóa, giúp phân tách rõ ràng trách nhiệm của từng thành phần phần mềm (Separation of Concerns):



Hình 3. 10: Sơ đồ minh họa cho Kiến trúc 3 tầng (3-tier Architecture)

Giải thích chức năng của từng tầng kiến trúc:

1. **Tầng Trình bày (Presentation Tier - Client-side):**

Website Lập Lịch và Quản Lý Công Việc

- Được xây dựng trên nền tảng **ReactJS** sử dụng công cụ build siêu tốc **Vite**. Tầng này chịu trách nhiệm hiển thị giao diện đồ họa tương tác cho người dùng, xử lý các hiệu ứng chuyển đổi mượt mà và quản lý trạng thái hiển thị của các view.
- **Zustand** được chọn làm giải pháp quản lý State toàn cục siêu nhẹ cho các dữ liệu ít thay đổi nhưng dùng ở nhiều nơi như thông tin phiên đăng nhập của User, trạng thái đóng mở Sidebar hoặc Modal Form.
- **TanStack Query (React Query)** quản lý trạng thái Server State. Nó đảm nhận việc đồng bộ hóa dữ liệu ngầm, caching thông tin các task để giao diện luôn phản hồi tức thì mà không cần hiển thị vòng quay loading liên tục.
- **Axios** đóng vai trò là HTTP Client, được cấu hình với interceptor tự động lấy JWT Token từ `localStorage` đính kèm vào header của mọi request gửi lên server.

2. Tầng Logic Nghiệp vụ (Business Logic Tier - Server-side):

- Chạy trên môi trường **Node.js** sử dụng framework **Express.js**. Đây là trung tâm đầu não điều khiển luồng hoạt động của toàn hệ thống.
- **Controllers** chứa mã nguồn xử lý logic nghiệp vụ thực tế của các tài nguyên: đăng nhập, thêm/sửa/xóa công việc, truy vấn thông tin AI.
- **JWT Middleware** hoạt động như một lớp bảo vệ bên ngoài các route bảo mật. Mọi request muốn chạm vào dữ liệu công việc bắt buộc phải đi qua đây để xác thực token.
- **Automation Service** là một background worker riêng biệt. Chạy song song và định kỳ mà không làm tắc nghẽn luồng xử lý HTTP request chính (Non-blocking I/O). Service này chịu trách nhiệm gọi tích hợp các dịch vụ bên thứ ba như IMAP Gmail để đồng bộ email, NVIDIA NIM AI để phân tích thực thể công việc và Discord Webhook để gửi cảnh báo.

3. Tầng Dữ liệu (Data Tier - Persistence):

- Cơ sở dữ liệu quan hệ **MySQL** lưu trữ vĩnh viễn toàn bộ thực thể của hệ thống.
- Backend kết nối tới cơ sở dữ liệu thông qua thư viện `mysql2` hỗ trợ cơ chế Connection Pool. Cơ chế này tự động duy trì một nhóm các kết nối rảnh rỗi và tái sử dụng chúng, giảm thiểu tối đa thời gian trễ của quá trình bắt tay thiết lập kết nối TCP mới cho mỗi truy vấn API.

CHƯƠNG 4: KẾT QUẢ TRIỂN KHAI WEBSITE

4.1 Môi trường triển khai

4.1.1 Công cụ phát triển

Hệ thống được phát triển trên môi trường cục bộ với các công cụ chính: **Node.js 18+** làm runtime JavaScript phía server, **npm 9+** để quản lý package, **MySQL 8.0** làm hệ quản trị cơ sở dữ liệu, **Vite 5.x** làm build tool cho React frontend, **Git 2.x** để quản lý phiên bản mã nguồn và **VS Code** làm môi trường phát triển tích hợp (IDE).

4.1.2 Dependencies chính

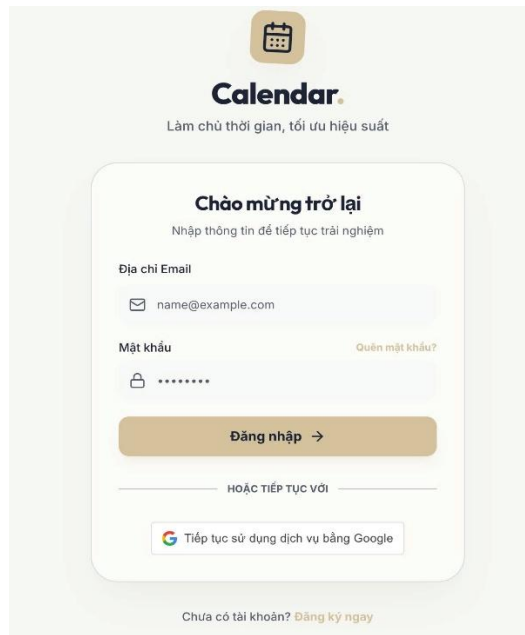
Frontend (React): Ứng dụng phía client được xây dựng trên nền **react** và **react-dom** làm thư viện UI cốt lõi, kết hợp **react-router-dom** để điều hướng phía client theo mô hình SPA. Quản lý trạng thái được chia thành hai lớp: **@tanstack/react-query** xử lý server state và caching dữ liệu từ API, **zustand** quản lý UI state toàn cục. Các thư viện hỗ trợ bổ sung gồm **axios** để gọi HTTP, **dayjs** để xử lý ngày tháng, **lucide-react** cung cấp bộ icon SVG, **sonner** hiển thị toast notification và **@react-oauth/google** tích hợp đăng nhập Google OAuth.

Backend (Node.js/Express): Phía server sử dụng **express** làm web framework chính, **mysql2** để kết nối cơ sở dữ liệu với hỗ trợ Promise. Bảo mật và xác thực được xử lý bởi **jsonwebtoken** để tạo và kiểm tra JWT, **bcryptjs** để mã hóa mật khẩu, **cors** để xử lý Cross-Origin requests. Các tích hợp ngoài bao gồm **imap-simple** và **mailparser** để kết nối và phân tích nội dung email qua IMAP, **google-auth-library** để xác thực Google ID Token. Ngoài ra, **morgan** hỗ trợ logging HTTP request và **dotenv** quản lý biến môi trường.

4.2 Kết quả giao diện các trang chính

4.2.1 Trang Đăng nhập

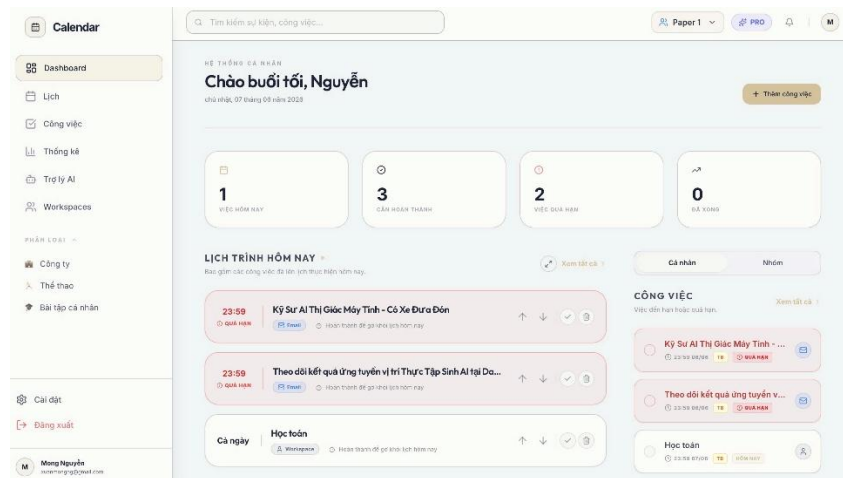
Trang đăng nhập được thiết kế tối giản, hiện đại theo phong cách Glassmorphism với card bo góc, hiệu ứng đổ bóng tạo cảm giác nổi. Giao diện bao gồm form nhập email và mật khẩu, nút đăng nhập chính và tùy chọn đăng nhập nhanh qua Google OAuth. Khi đang xử lý, nút chuyển sang trạng thái loading với spinner để phản hồi người dùng.



Hình 4. 1: Giao diện trang Đăng nhập

4.2.2 Trang Dashboard

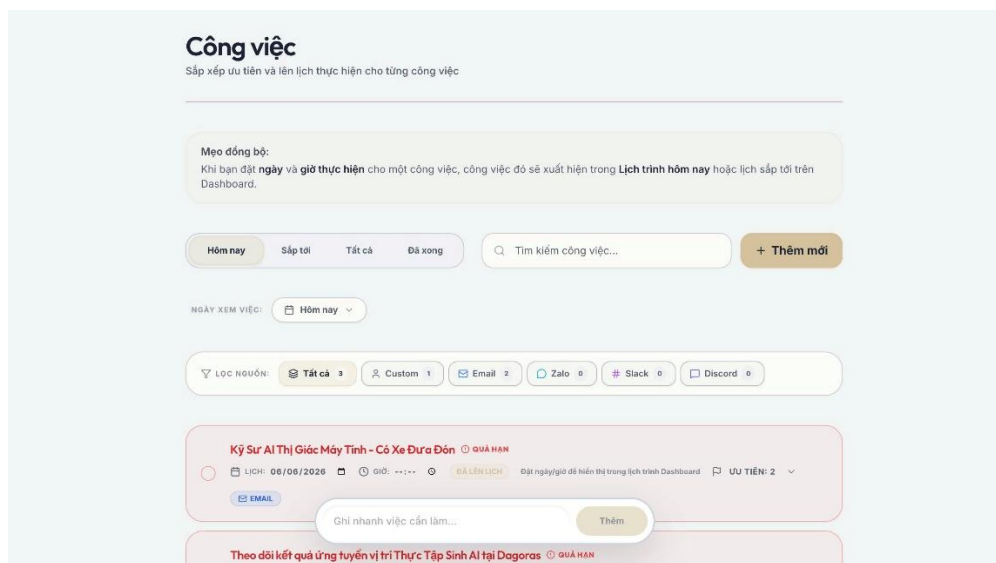
Dashboard là trang tổng quan chính sau khi đăng nhập, cung cấp cái nhìn toàn cảnh về tình trạng công việc trong ngày. Phần trên hiển thị lời chào theo buổi kèm ngày tháng tiếng Việt và 4 card thống kê nhanh: số việc hôm nay, cần hoàn thành, quá hạn và đã xong. Phần dưới chia thành hai cột: cột chính hiển thị lịch trình theo giờ dạng Timeline, cột phụ liệt kê các công việc đến hạn hôm nay theo mức độ ưu tiên. Góc dưới hiển thị 3 thông báo gần nhất với badge chấm xanh cho thông báo chưa đọc.



Hình 4. 2: Giao diện trang Dashboard

4.2.3 Trang Quản lý công việc

Đây là trang phức tạp nhất trong hệ thống, cung cấp đầy đủ các thao tác quản lý task. Người dùng có thể điều hướng qua 4 tab: Hôm nay, Sắp tới, Tất cả và Đã xong. Thanh công cụ phía trên hỗ trợ tìm kiếm và lọc task theo ngày, theo nguồn gốc (Custom, Email, Zalo, Slack, Discord). Mỗi task trong danh sách cho phép chỉnh sửa tiêu đề trực tiếp, đặt ngày giờ bằng picker inline, chọn mức ưu tiên và đánh dấu hoàn thành chỉ bằng một cú click. Cuối trang có thanh nhập nhanh cố định để thêm task tức thì mà không cần mở modal.

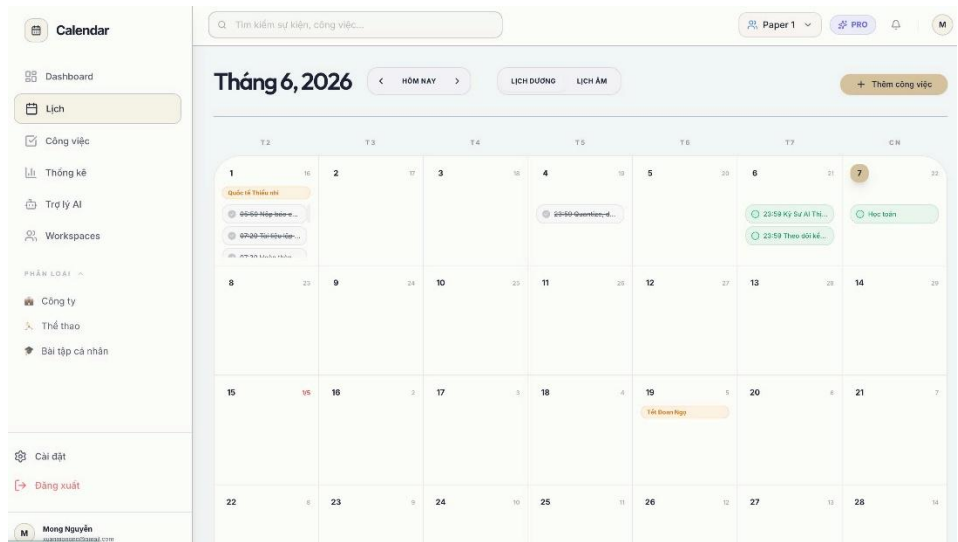


Hình 4. 3: Giao diện trang Quản lý công việc

4.2.4 Trang Lịch

Trang lịch hiển thị toàn bộ công việc dưới dạng lưới tháng 7 cột $\times$ 6 tuần. Mỗi ô ngày thể hiện đồng thời ngày dương lịch, ngày âm lịch và danh sách task trong ngày. Ngày hiện tại được đánh dấu bằng badge tròn nổi bật. Các ngày lễ quốc gia và ngày kỷ niệm được gắn nhãn màu riêng biệt. Người dùng có thể chuyển đổi giữa chế độ hiển thị Dương lịch và Âm lịch làm số chính trên mỗi ô, đồng thời click vào ngày bất kỳ để xem toàn bộ task của ngày đó.

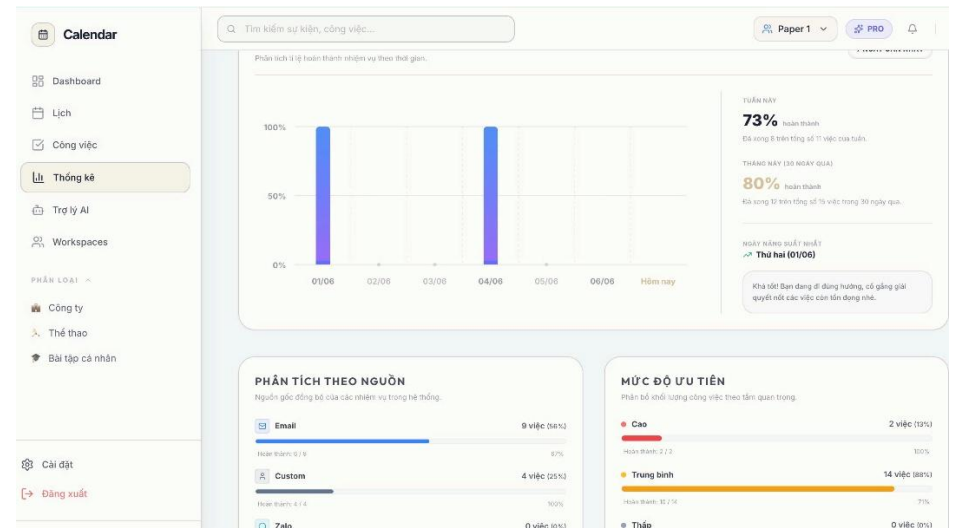
Website Lập Lịch và Quản Lý Công Việc



Hình 4. 4: Giao diện trang Lịch

4.2.5 Trang Phân tích hiệu suất

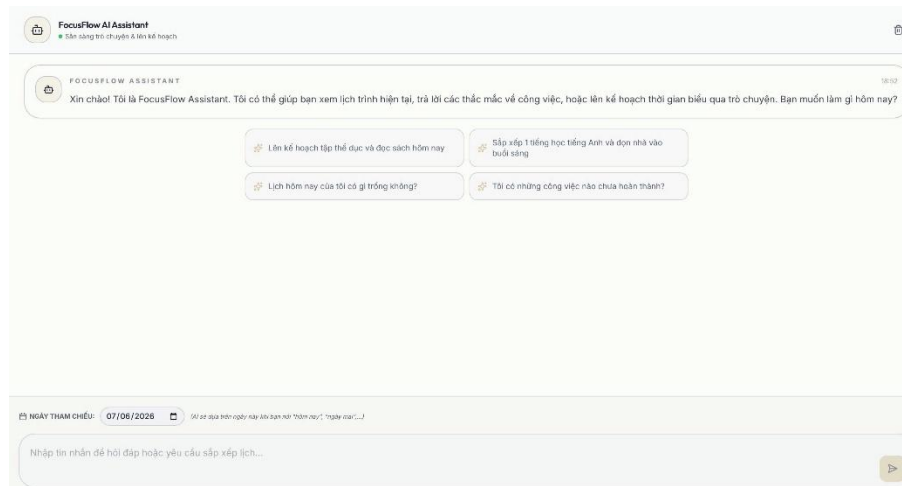
Trang Analytics cung cấp góc nhìn định lượng về năng suất cá nhân thông qua các biểu đồ SVG tự phát triển. Card chính hiển thị biểu đồ cột gradient 7 ngày gần nhất với tooltip chi tiết khi hover, kèm panel thống kê tỉ lệ hoàn thành theo tuần và theo 30 ngày, cùng gợi ý thông minh dựa trên hiệu suất thực tế. Hai card phụ phân tích phân bổ công việc theo nguồn gốc và theo mức độ ưu tiên dưới dạng biểu đồ thanh ngang.



Hình 4. 5: Giao diện trang Phân tích hiệu suất

4.2.6 Trang Trợ lý AI

Giao diện chat trực quan giống ứng dụng nhắn tin, với tin nhắn của AI hiển thị bên trái và tin nhắn người dùng bên phải. Khi trợ lý AI đề xuất lịch trình, hệ thống tự động render một thẻ kế hoạch trực quan liệt kê các task đề xuất kèm giờ, mức ưu tiên và lý do sắp xếp — người dùng chỉ cần bấm "Chấp nhận" để lưu toàn bộ vào hệ thống. Phía dưới khung chat có các chip gợi ý câu hỏi nhanh để người dùng bắt đầu tương tác dễ dàng hơn.

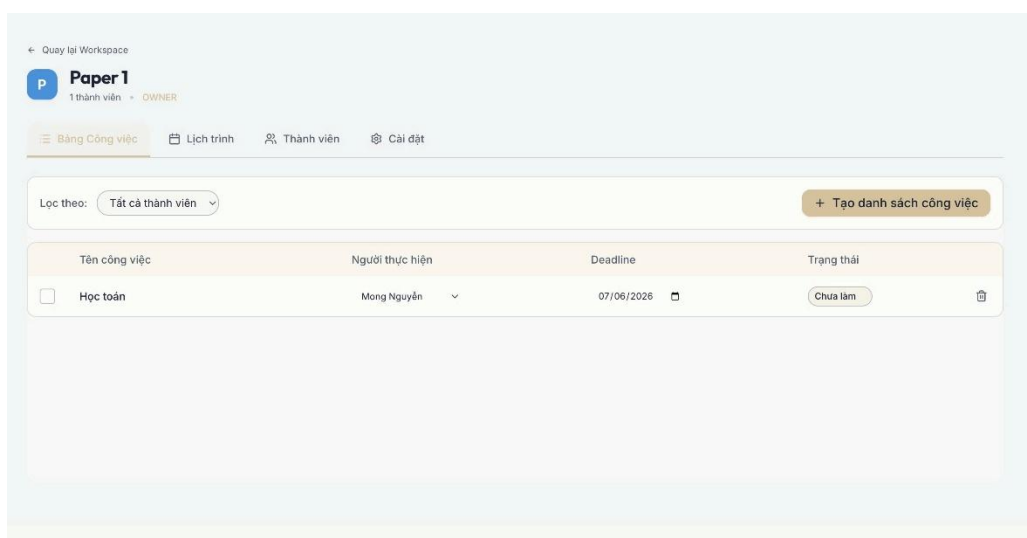


Hình 4. 6: Giao diện trang Trợ lý AI

4.2.7 Trang Không gian làm việc nhóm

Tính năng Workspace cho phép người dùng tạo và quản lý các không gian làm việc chung. Trang danh sách hiển thị các workspace dưới dạng card với vai trò của người dùng (Owner, Admin, Member) và màu đại diện riêng. Trong mỗi workspace, bảng công việc nhóm cho phép Admin/Owner giao việc cho từng thành viên qua dropdown inline và tạo hàng loạt task cùng lúc. Thành viên thường chỉ thấy và thao tác trên các công việc được giao cho mình. Trang quản lý thành viên hỗ trợ mời người qua email và phân quyền trực tiếp trên giao diện.

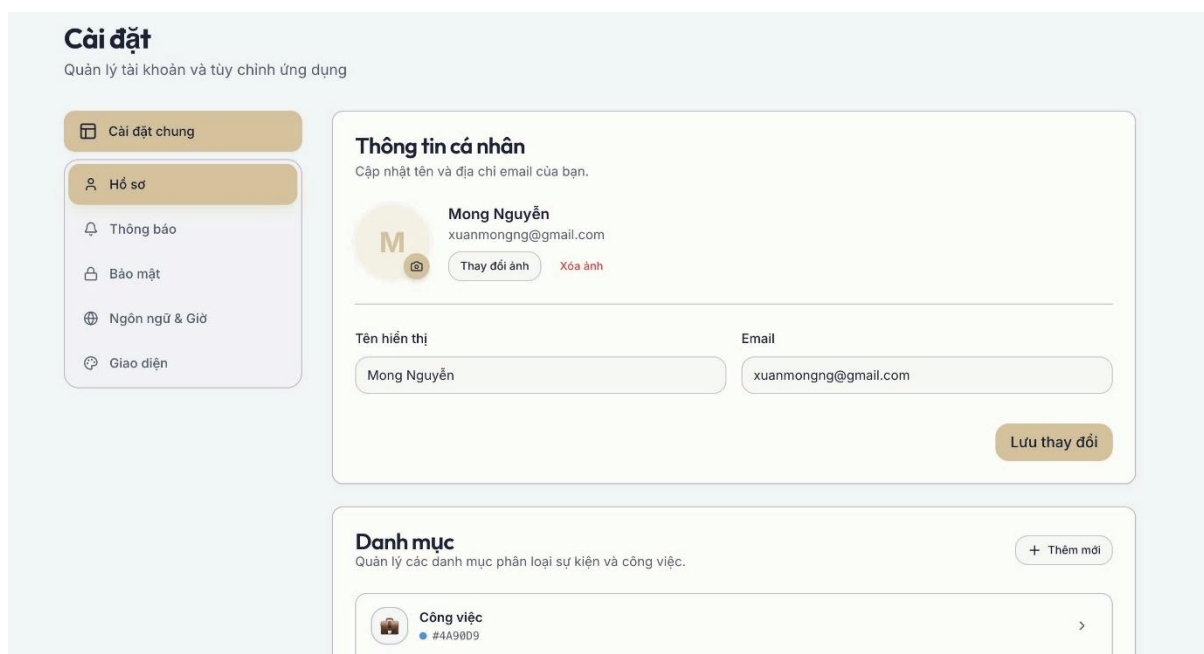
Website Lập Lịch và Quản Lý Công Việc



Hình 4. 7: Giao diện trang Không gian làm việc nhóm

4.2.8 Trang Cài đặt

Trang cài đặt được tổ chức theo dạng sidebar tab với 5 mục: Hồ sơ, Thông báo, Bảo mật, Ngôn ngữ & Giờ và Giao diện. Panel Hồ sơ cho phép chỉnh sửa tên, ảnh đại diện và danh mục cá nhân. Panel Thông báo cung cấp 4 toggle bật/tắt cho từng loại nhắc nhở. Panel Giao diện cho phép chọn theme Sáng, Tối hoặc theo hệ thống.



Hình 4. 8: Giao diện trang Cài đặt

4.3 Kết quả chức năng tự động hóa nền

4.3.1 Dịch vụ đồng bộ email tự động

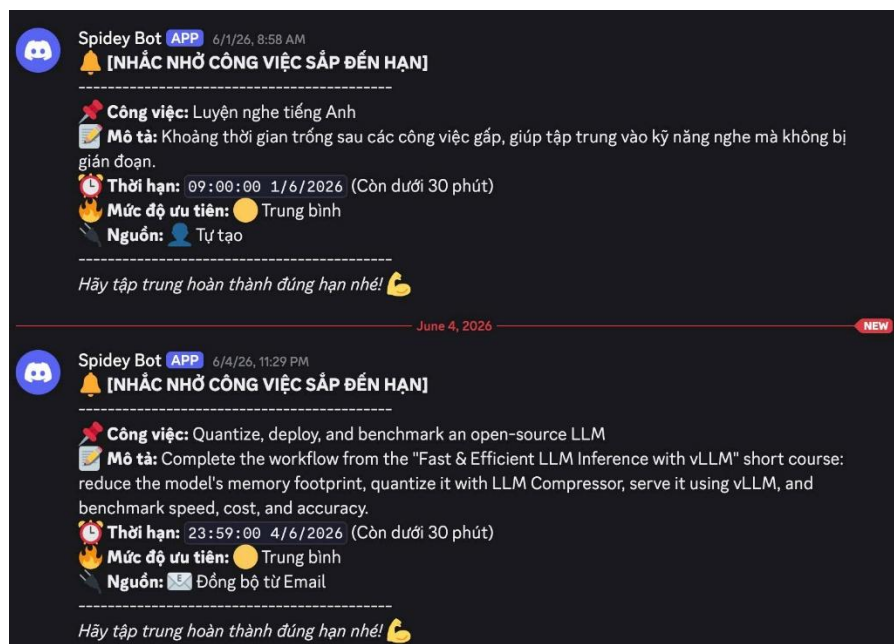
Khi hệ thống khởi động, dịch vụ tự động hóa kết nối IMAP đến hộp thư của người dùng, quét thư chưa đọc và lọc qua bộ lọc thông minh 4 lớp để loại bỏ thư rác và giữ lại email công việc thực sự. Các email hợp lệ được chuyển cho AI phân tích và trích xuất thông tin task, sau đó tự động lưu vào hệ thống kèm thông báo để người dùng xác nhận.

```
...
[✓] Kết nối Database MySQL thành công!
[Task Cleanup] Không có dữ liệu cũ nào (> 30 ngày) cần dọn dẹp.
[Email Sync] Đang tìm kiếm thư chưa đọc...
[Email Sync] Tìm thấy 0 thư mới.
[Email Sync] Hoàn thành quét thư.
```

Hình 4. 9: Log kết quả đồng bộ email tự động

4.3.2 Dịch vụ nhắc nhở qua Discord

Mỗi phút, hệ thống tự động rà soát các task sắp đến hạn trong vòng 30 phút và gửi tin nhắn nhắc nhở định dạng Markdown trực tiếp vào kênh Discord cá nhân của người dùng. Mỗi thông báo hiển thị đầy đủ tên công việc, mô tả, thời hạn, mức độ ưu tiên và nguồn gốc task. Mỗi task chỉ được gửi thông báo đúng một lần.

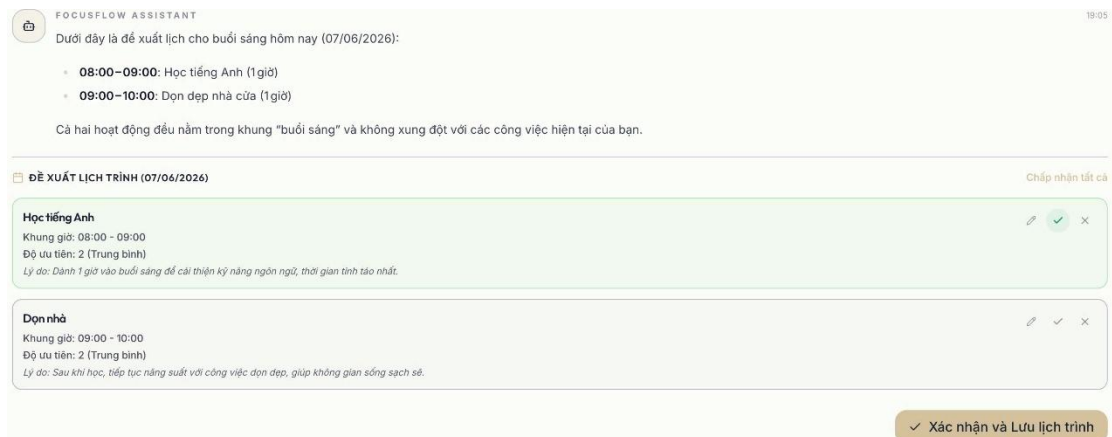


Hình 4. 10: Thông báo nhắc nhở deadline qua Discord

4.4 Kết quả tích hợp AI

4.4.1 AI đề xuất lịch trình

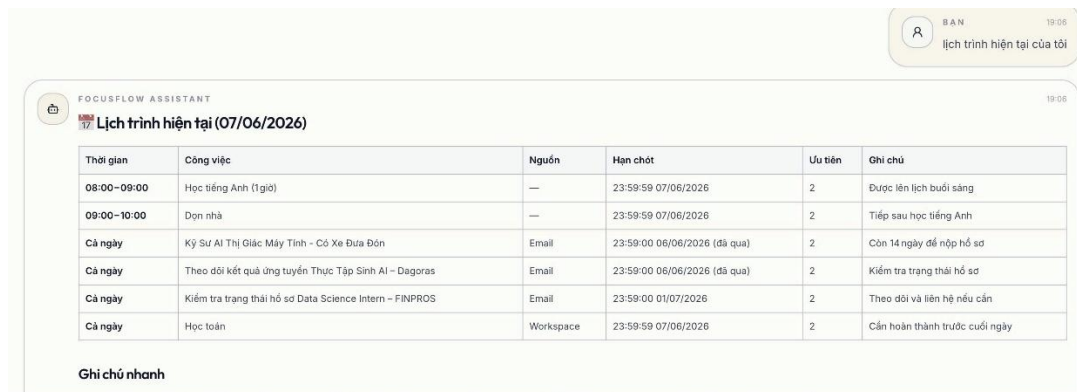
Khi người dùng yêu cầu lên kế hoạch bằng tiếng Việt tự nhiên, AI phân tích nhu cầu và trả về lịch trình chi tiết dưới dạng thẻ kế hoạch trực quan với từng khung giờ, mức ưu tiên và lý do sắp xếp cụ thể. Người dùng xem xét và chấp nhận hoặc từ chối toàn bộ kế hoạch chỉ bằng một thao tác.



Hình 4. 11: AI đề xuất lịch trình dạng thẻ kế hoạch

4.4.2 AI trích xuất task từ email

Khi nhận được email công việc, AI tự động nhận diện và trích xuất các trường thông tin quan trọng bao gồm tiêu đề task, mô tả, deadline và mức độ ưu tiên — chuyển nội dung email thô thành task có cấu trúc sẵn sàng lưu vào hệ thống mà không cần người dùng can thiệp thủ công.



Hình 4. 12: Kết quả AI trích xuất task từ nội dung email

4.5 Hạn chế và hướng phát triển tương lai

4.5.1 Hạn chế hiện tại

- Bên cạnh các kết quả đạt được, hệ thống FocusFlow vẫn còn một số hạn chế cần ghi nhận. Thứ nhất, ứng dụng yêu cầu kết nối mạng liên tục và chưa hỗ trợ chế độ hoạt động ngoại tuyến (offline mode). Thứ hai, các tính năng AI phụ thuộc hoàn toàn vào NVIDIA NIM API bên ngoài, nếu dịch vụ này gián đoạn thì toàn bộ chức năng AI sẽ ngừng hoạt động. Thứ ba, mã nguồn hiện chưa có unit test và integration test cho các controller và service, làm giảm độ tin cậy khi bảo trì và mở rộng. Thứ tư, giao diện mới ở mức responsive cơ bản, chưa được tối ưu theo hướng mobile-first và chưa có ứng dụng di động native.

4.5.2 Hướng phát triển tương lai

- Trong thời gian tới, hệ thống có thể được mở rộng theo nhiều hướng. Về kênh tích hợp, có thể bổ sung Telegram Bot API và Zalo OA API cho nhắc nhở đa nền tảng, đồng thời tích hợp Google Calendar API để đồng bộ lịch hai chiều. Về năng lực AI, hướng phát triển tiếp theo bao gồm nâng cấp lên các mô hình mạnh hơn, xây dựng cơ chế AI tự học thói quen người dùng để gợi ý lịch trình cá nhân hóa và tích hợp voice command để tạo task bằng giọng nói. Về tính năng cộng tác nhóm, có thể bổ sung Real-time collaboration qua WebSocket để đồng bộ trạng thái công việc tức thì, kèm theo tính năng bình luận và đính kèm tệp trực tiếp vào từng task nhóm. Về hạ tầng kỹ thuật, hướng ưu tiên gồm đóng gói Docker để dễ triển khai, thiết lập CI/CD pipeline qua GitHub Actions, tích hợp hệ thống monitoring và logging, cũng như bổ sung đầy đủ unit test và integration test cho toàn bộ hệ thống.

KẾT LUẬN

Đề tài "FocusFlow — Hệ thống quản lý lịch trình và công việc cá nhân tích hợp AI" đã được nghiên cứu và triển khai thành công dưới dạng một ứng dụng web hoàn chỉnh, giải quyết trực tiếp bài toán quản lý thời gian và công việc trong bối cảnh thông tin phân tán ngày càng phức tạp của người dùng hiện đại.

Về mặt kỹ thuật, đề tài đã xây dựng thành công kiến trúc web 3 tầng với Frontend React kết hợp Backend Node.js/Express và cơ sở dữ liệu MySQL, đồng thời tích hợp hiệu quả nhiều công nghệ và giao thức thực tế bao gồm IMAP, Google OAuth 2.0, Discord Webhook và NVIDIA NIM API. Qua đó, nhóm đã làm chủ được kỹ thuật Prompt Engineering để ép mô hình ngôn ngữ lớn trả về dữ liệu có cấu trúc phục vụ logic nghiệp vụ, cũng như thiết kế các background worker xử lý tác vụ bất đồng bộ an toàn và ổn định.

Về mặt sản phẩm, FocusFlow đã hiện thực hóa đầy đủ các tính năng cốt lõi đề ra: tự động trích xuất công việc từ email bằng AI, trợ lý AI lên kế hoạch bằng tiếng Việt tự nhiên, nhắc nhở deadline chủ động qua Discord, phân tích hiệu suất cá nhân trực quan và không gian cộng tác nhóm tích hợp liền mạch. Sản phẩm có giao diện hiện đại, kiến trúc mở và sẵn sàng vận hành thực tế trên môi trường cục bộ.

Mặc dù vẫn còn một số hạn chế về mobile, offline mode và độ phủ kiểm thử, đây là nền tảng vững chắc để tiếp tục phát triển và mở rộng trong tương lai. Đề tài không chỉ mang lại giá trị thực tiễn cho người dùng mà còn là cơ hội quý báu để tích lũy kinh nghiệm xây dựng hệ thống phần mềm đầy đủ từ thiết kế, lập trình đến tích hợp AI trong một sản phẩm hoàn chỉnh.

TÀI LIỆU THAM KHẢO

- [1] Trường Đại học Sư phạm Kỹ thuật Hưng Yên. Giáo trình Công nghệ Web và Ứng dụng. Khoa Công nghệ Thông tin.
- [2] Notion Labs, Inc. Notion – Không gian làm việc AI của bạn. Available: <https://www.notion.com/vi>. Accessed: Jun. 07, 2026.
- [3] ClickUp. ClickUp – Project Management and Productivity Platform. Available: <https://clickup.com/>. Accessed: Jun. 07, 2026.
- [4] Google. Google Calendar. Available: <https://calendar.google.com/>. Accessed: Jun. 07, 2026.
- [5] Google. Chọn cài đặt ứng dụng email IMAP của bạn cho Gmail. Gmail Help. Available: <https://support.google.com/mail/answer/78892?hl=vi>. Accessed: Jun. 07, 2026.
- [6] Meta Open Source. React Documentation – Quick Start. Available: <https://react.dev/learn>. Accessed: Jun. 07, 2026.
- [7] REST API Tutorial. REST API Tutorial. Available: <https://www.restapitutorial.com/>. Accessed: Jun. 07, 2026.
- [8] OpenJS Foundation. Introduction to Node.js. Node.js Learn. Available: <https://nodejs.org/learn/getting-started/introduction-to-nodejs>. Accessed: Jun. 07, 2026.